

Towards Secure MLOps: Surveying Attacks, Mitigation Strategies, and Research Challenges

RAJ PATEL*, The University of Alabama, USA

HIMANSHU TRIPATHI, Mississippi State University, USA

JASPER STONE, Mississippi State University, USA

NOORBAKHS AMIRI GOLILARZ, The University of Alabama, USA

SUDIP MITTAL, The University of Alabama, USA

SHAH RAM RAHIMI, The University of Alabama, USA

VINI CHAUDHARY, Mississippi State University, USA

The rapid adoption of machine learning (ML) technologies has driven organizations across diverse sectors to seek efficient and reliable methods to accelerate model development-to-deployment. Machine Learning Operations (MLOps) has emerged as an integrative approach addressing these requirements by unifying relevant roles and streamlining ML workflows. As the MLOps market continues to grow, securing these pipelines has become increasingly critical. However, the unified nature of MLOps ecosystem introduces vulnerabilities, making them susceptible to adversarial attacks where a single misconfiguration can lead to compromised credentials, severe financial losses, damaged public trust, and the poisoning of training data. Our paper presents a systematic application of the MITRE ATLAS (Adversarial Threat Landscape for Artificial-Intelligence Systems) framework, supplemented by reviews of white and grey literature, to systematically assess attacks across different phases of the MLOps ecosystem. We begin by reviewing prior work in this domain, then present our taxonomy and introduce a threat model that captures attackers with different knowledge and capabilities. We then present a structured taxonomy of attack techniques explicitly mapped to corresponding phases of the MLOps ecosystem, supported by examples drawn from red-teaming exercises and real-world incidents. This is followed by a taxonomy of mitigation strategies aligned with these attack categories, offering actionable early-stage defenses to strengthen the security of MLOps ecosystem. Given the gradual evolution and adoption of MLOps, we further highlight key research gaps that require immediate attention. Our work emphasizes the importance of implementing robust security protocols from the outset, empowering practitioners to safeguard MLOps ecosystem against evolving cyber attacks.

CCS Concepts: • **Computing methodologies** → **Artificial intelligence; Machine learning**; • **Security and privacy**;

Additional Key Words and Phrases: Machine Learning Operations (MLOps), Cyberattack

1 Introduction

Machine Learning (ML) technologies have fueled remarkable growth across diverse industries, prompting organizations to swiftly transition models from development into production to secure competitive advantages.

Authors' Contact Information: Raj Patel, rpatel38@ua.edu, The University of Alabama, Tuscaloosa, Alabama, USA; Himanshu Tripathi, Mississippi State University, Mississippi State, USA, ht557@msstate.edu; Jasper Stone, Mississippi State University, Mississippi State, USA, jws819@msstate.edu; Noorbakhsh Amiri Golilarz, noor.amiri@ua.edu, The University of Alabama, Tuscaloosa, Alabama, USA; Sudip Mittal, sudip.mittal@ua.edu, The University of Alabama, Tuscaloosa, Alabama, USA; Shahram Rahimi, srahimi1@ua.edu, The University of Alabama, Tuscaloosa, Alabama, USA; Vini Chaudhary, vchaudhary@cse.msstate.edu, Mississippi State University, Mississippi State, USA.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, or post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 3068-3564/2026/3-ART

<https://doi.org/10.1145/3796728>

To manage this acceleration, companies are increasingly adopting Machine Learning Operations (MLOps) lifecycle framework [49]. First introduced in 2015 [231], MLOps is defined as “a set of practices designed to create an assembly line for building and running machine learning models. It helps companies automate tasks and deploy models quickly, ensuring everyone involved (data scientists, engineers, IT) can cooperate smoothly and monitor and improve models for better accuracy and performance” [241]. The MLOps market reached \$2.2B in 2024 [49], with projections indicating a 43% expansion over the next five years [238]. Industry adoption is evidenced by full MLOps implementations at major companies including Netflix, Uber, DoorDash, and Lush [216]. Major cloud providers and vendors now offer integrated MLOps platforms supporting data ingestion, training, deployment, and monitoring at enterprise scale [52]. These developments collectively demonstrate that MLOps has become a central coordinating layer in modern AI-enabled infrastructures.

The strength of MLOps arises from combining the iterative nature of ML with the Continuous Integration and Continuous Delivery (CI/CD) practices established by DevOps. DevOps refers to “a set of practices for automating the processes between software development and information technology operations teams so that they can build, test, and release software faster and more reliably. The goal is to shorten the systems development life cycle and improve reliability while delivering features, fixes, and updates frequently in close alignment with business objectives” [212]. Integrating DevOps methodologies with ML workflows results in robust and flexible MLOps ecosystem capable of rapidly responding to evolving business and security requirements.

Although MLOps offers substantial benefits, the pressure to accelerate model deployment for competitive gain often results in insufficient attention to security, increasing exposure to vulnerabilities across the MLOps ecosystem. Research by IBM X-Force demonstrates that widely used enterprise MLOps platforms, including Azure Machine Learning, BigML, and Google Cloud Vertex AI, are vulnerable to attacks by adversaries with valid credentials, enabling training data poisoning, model and training data theft, and exploitation of production pipelines and connected data lakes [52], exposing critical Machine Learning Security Operations (MLSecOps) concerns [21]. With the MLOps market projected to grow 43% over the next five years [49, 238], securing these pipelines has become critical. High-profile incidents such as the ShadowRay vulnerability [71], where insufficient authentication control mechanisms allowed attackers to seize compute resources and exfiltrate more than \$1 billion worth of data in September 2023, underline the severity of these risks. Similarly, the Arbitrary Code Execution incident involving Google Colab [1] highlighted the dangers of malicious scripts hidden in shared Jupyter Notebooks, emphasizing the urgent need for rigorous code review processes. DeepSeek’s V3 model [29] allegedly resulted from repeated unauthorized distillation queries to OpenAI’s models [74], demonstrating intellectual property theft risks. Additionally, research conducted by SpiderSilk [253] demonstrated how a single misconfiguration compromised critical credentials, leading to unauthorized access to Clearview AI’s sensitive code repositories. Given that Clearview AI’s facial recognition tools are used in matching identities using publicly available images, the implications of such a breach are especially concerning. These examples collectively highlight how security lapses in the MLOps ecosystem can lead to infiltration, data theft, operational disruption, and substantial reputational damage.

Traditional security measures, such as deploying standard malware or virus detection tools within MLOps ecosystem, are insufficient to counter these evolving attacks. Adversaries continuously adapt and refine their tactics, employing defense-aware strategies capable of evading traditional security measures. Skylight’s findings [236] demonstrated how adversarial inputs could evade current malware detection systems. In 2020, Palo Alto Networks’ Security AI research team [252] showcased how generic domain name mutation techniques could defeat Convolutional Neural Network (CNN)-based DGA detection modules. Such mutations evade most ML-based detection systems, underscoring the importance of thorough robustness evaluation before deployment. These attacks often leverage Open Source Intelligence Gathering (OSINT) [64] to maximize infiltration rates while minimizing detection and attribution. Such examples underscore the need to embed security deeply within the MLOps ecosystem, addressing threats at every stage of the ML lifecycle.

In light of these gaps, our study is guided by the following research questions:

- **RQ1:** What resources exist that systematically catalog AI-related attacks and attacker techniques, and how can they be leveraged to comprehensively map vulnerabilities within the MLOps ecosystem?
- **RQ2:** What is the severity of the threats presented to MLOps ecosystem, and how many of these threats have been observed in real-world incidents (or case studies), as opposed to remaining theoretical in nature?
- **RQ3:** How well do existing adversarial frameworks align their proposed defenses and mitigations with the identified attack techniques, and to what extent do they provide comprehensive coverage of this attack surface?
- **RQ4:** What notable gaps persist, and what further research is necessary to enhance security across the MLOps ecosystem?

To address these research questions and challenges, this paper provides the first comprehensive mapping of MLOps-centric attacks using the MITRE ATLAS (Adversarial Threat Landscape for Artificial Intelligence Systems) framework [208], an exhaustive and continuously updated catalog of AI-centric attack lifecycles. Additionally, our analysis integrates established frameworks such as MITRE ATT&CK [209] and the emerging OWASP (Open Worldwide Application Security Project) Top 10 for Large Language Models [218] with extensive white and grey literature, including security research papers, industry reports, documented incidents, and red teaming exercises. This multi-source approach enables us to identify key security risks and validate attack techniques through real-world evidence. By integrating these resources, we enhance our attack assessment and address existing gaps within the MITRE ATLAS framework. Leveraging the ATLAS framework enables organizations to perform structured evaluations of attack vectors, uncover vulnerabilities, and integrate security measures throughout the MLOps ecosystem. Our extensive review of mitigation strategies further assists organizations in protecting mission-critical assets, reinforcing resilience against continuously evolving attacks. The major contributions of this paper are as follows:

- We highlight how adversaries exploit vulnerabilities at each phase of the MLOps ecosystem using the ATLAS framework together with relevant white and grey literature to identify potential attack vectors.
- We review a comprehensive taxonomy of MLOps-centric attacks and validate their applicability through examples drawn from real-world incidents and/or red-team exercises documented in the ATLAS framework.
- We present a structured taxonomy of mitigation strategies to enhance the detection and prevention of adversarial attacks on MLOps ecosystem.
- We discuss challenges and provide research recommendations to enhance the security and robustness of the MLOps ecosystem, ensuring resilience against emerging sophisticated cyberattacks.

The remainder of this paper presents a comprehensive analysis of adversarial risks inherent to MLOps ecosystems. Section II introduces the MLOps ecosystem, detailing foundational concepts, operational taxonomy, and real-world implementations. Section III reviews prior work in this domain and explains how our study is positioned, then sets out the survey taxonomy and defines a threat model that captures attackers with different motivations, knowledge, and capabilities. Section IV leverages the MITRE ATLAS framework to review the evolving attack landscape, illustrating the impact of these threats through empirical evidence from real-world incidents and red-teaming exercises. Section V presents mitigation strategies addressing systemic vulnerabilities within the MLOps ecosystem. Finally, Section VI outlines ongoing research challenges in securing MLOps and identifies opportunities to strengthen defenses through integration of emerging security technologies.

2 MLOps Overview

The concept of Machine Learning Operations (MLOps) emerged in 2015 [231] to bridge the gap between isolated model development to practical deployment. MLOps enhances automation, communication, and monitoring within ML workflow that are inspired by DevOps principles, aligning them with organization's objective and creating tangible value [242]. Large Language Model Operations (LLMOps) is a specialized subset of MLOps where replacing model, data, and evaluation blocks with LLM oriented versions preserves the pipeline structure [47, 238].

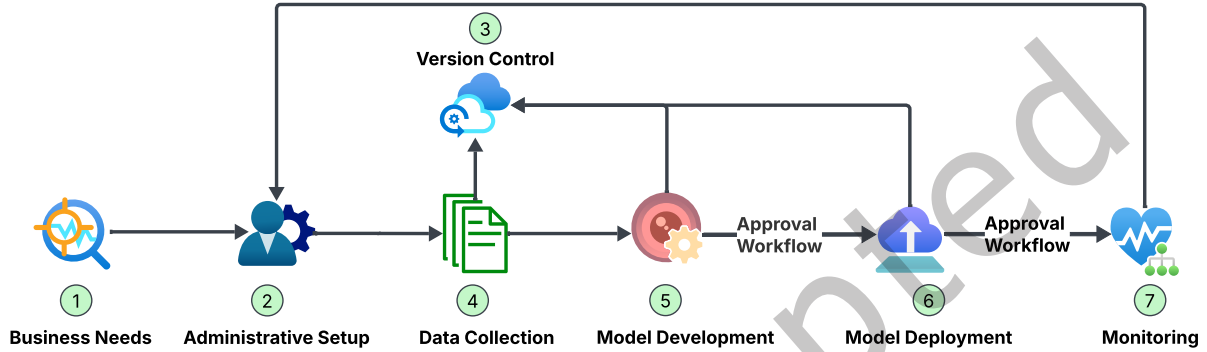


Fig. 1. The MLOps ecosystem begins by identifying business needs, followed by administrative setup, model development, and deployment. Continuous monitoring then provides essential feedback, enabling iterative improvements.

Fig. 1 provides a visual summary of the MLOps ecosystem, which is explored in greater depth in [238]. Here, we outline the key phases most relevant to the security considerations discussed in the following sections. The MLOps process begins with the clear identification of business needs and the establishment of specific Objectives and Key Results (OKRs) that guide decision-making and operational priorities [239] [31]. Effective machine learning models can directly influence commercial outcomes, and a scalable MLOps ecosystem enables the continuous training and deployment of projects as they move from proof-of-concept to production scale [211].

Following business alignment, data collection uses many methods including web scraping, APIs, IoT sensors, and crowdsourcing [211, 242, 251]. Augmentation and transfer learning can enhance the data that is readily available in situations where information is limited [62, 257]. Data preparation typically begins with cleaning procedures that address missing values, errors, and duplicates. This is followed by normalization techniques to ensure data consistency and enhance algorithmic efficiency [63, 251]. Dataset reliability is improved through robust versioning and seamless data integration, which in turn support effective feature engineering and exploratory data analysis (EDA) [211].

A robust administrative framework underpins scalable MLOps infrastructure by supporting key operations such as resource provisioning, user role management, and system monitoring [238]. It ensures access to necessary computational resources, including CPUs, GPUs, and TPUs, for efficient training and deployment. Monitoring tools track system health and anomalies to maintain model consistency and integrity. Additionally, well-defined roles, permissions, and Identity and Access Management (IAM) controls [7] promote secure collaboration and safeguard against unauthorized access.

The model development phase focuses on transforming data into actionable insights through robust experimentation and iterative refinement. Key tasks include data formatting, exploratory data analysis (EDA), feature engineering, and algorithm selection. Using statistical and visual techniques, EDA uncovers patterns, relationships, and anomalies that guide effective feature selection [59, 69]. Feature engineering, through methods such

as dimensionality reduction and polynomial expansion, refines raw inputs to improve model accuracy and robustness [62, 211]. Algorithm selection, which includes methods such as decision trees, linear regression, and neural networks, is informed by both data characteristics and organizational needs [62, 239]. Rigorous evaluation metrics, including precision, F1-score, ROC-AUC, and MSE, along with considerations for model interpretability, help ensure alignment with strategic goals and build stakeholder trust [62, 239, 257]. To mitigate overfitting and improve inference, training procedures incorporate techniques such as early stopping, adaptive learning rates, and cross-validation [69, 257].

Version control and experiment tracking allow teams to document and monitor changes to data, features, hyperparameters, and source code, promoting consistency and reproducibility. These systems also provide rollback capabilities in case of performance degradation, helping maintain the stability and reliability of the MLOps workflow [62, 69]. Continuous Integration (CI) automates the testing and validation of new code and model updates, ensuring compatibility and minimizing deployment risks [41, 58].

The deployment phase emphasizes rigorous testing and validation under conditions that closely mirror the production environment, ensuring both reliability and performance. Quality assurance procedures such as batch processing and real-time inference validations are conducted to verify that models meet predefined accuracy and latency requirements [5]. Responsible AI practices are also incorporated at this stage to mitigate bias and promote fairness. Before release into production, models must pass through a gated approval workflow that includes a combination of automated performance evaluations and manual reviews to ensure compliance with robustness and security standards [238]. Once approved, models are deployed for end-user access and configured to support diverse inference scenarios. Benchmarking tools, including MLPerf [214], are used to assess production readiness and verify that inference quality is maintained in the target environment [225].

Continuous monitoring is essential in complex multi-cloud environments, enabling performance tracking across testing, staging, and production stages to ensure timely detection and resolution of deviations [38]. This comprehensive approach facilitates sustained alignment with evolving business needs.

In summary, the comprehensive MLOps ecosystem effectively manages the machine learning lifecycle, integrating continuous development, deployment, and monitoring. By prioritizing alignment with business objectives and responsible AI practices, organizations can rigorously evaluate and continuously monitor the performance of their machine learning systems. These efforts contribute to operational efficiency, ethical model behavior, sustained reliability, and overall trustworthiness. However, despite these advancements, MLOps ecosystem remain vulnerable to security threats. Ensuring long-term dependability and stakeholder trust requires ongoing research and integration of robust, context-aware security.

3 Survey Approach And Taxonomy

The preceding section examined the MLOps ecosystem and described how tools, roles, and software components interact to automate the machine learning lifecycle. This automation accelerates development and operational reliability but also inherits security exposure at each stage of the lifecycle, especially when common standards are missing and organizations combine heterogeneous platforms and tools [31]. In this section, we focus on how these security exposures manifest when viewing MLOps as an end-to-end socio-technical system rather than as a group of isolated components. To address RQ1, we draw on peer-reviewed research, technical blogs, industry reports, and public frameworks such as MITRE ATLAS and MITRE ATT&CK, to connect concrete attacker tactics and techniques to the main stages of the MLOps lifecycle [208, 209]. To our knowledge, this is the first survey to systematically map attacker techniques to the full MLOps and LLMOps lifecycle, and to use this mapping as the foundation for both a security taxonomy and a threat model. The remainder of this section explains how our work is situated in the literature, how we structure the MLOps ecosystem into three taxonomical families, and how we use this structure to define a threat model that reflects attacker behavior and operational concerns.

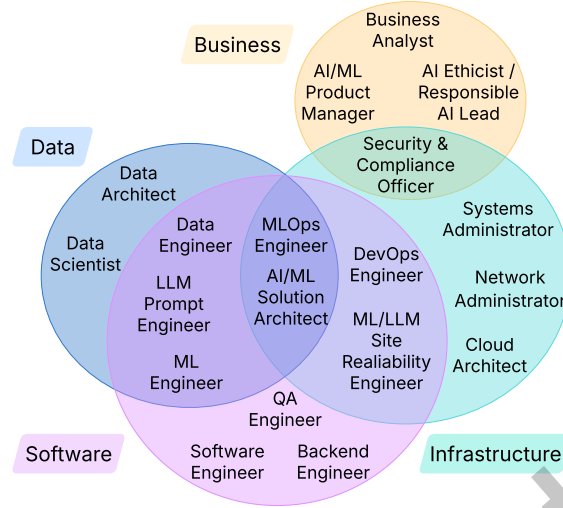


Fig. 2. Venn diagram of potential operational roles contributing to secure MLOps ecosystem development.

3.1 Survey Landscape and Positioning

The machine learning security research community has typically studied attacks and defenses by focusing on individual attack families such as poisoning attacks, privacy attacks, and evasion attacks, with an emphasis on model or data behavior in controlled settings. These studies provide insight into how specific techniques work and how models can fail. However, they rarely analyze these attacks within the context of the complete MLOps lifecycle, which spans design, development, deployment, and operations. As a result, most surveys consider each phase of the lifecycle as independent, even though decisions made during early design stages directly shape downstream risks. Only a limited number of works consider an interconnected perspective that takes into account how data pipelines, training workflows, deployment stacks, and monitoring systems share common infrastructure. In addition, most prior surveys approach the subject from a research or platform engineering viewpoint and give limited attention to the needs of end users and operators who are responsible for maintaining secure and reliable systems over time. In this subsection, we review representative surveys and frameworks on secure MLOps and SecMLOps, and clarify how our work is positioned both from a research perspective and from an operational perspective. This review motivates our focus on security across the entire lifecycle and prepares the ground for the taxonomy that follows.

Calefato et al. [21] synthesize academic studies and practitioner reports to create a catalog of risks and best practices for machine learning operations, covering access control, network protection, secure deployment, monitoring, privacy, supply chain integrity, and organizational culture. They identify recurring risks including unauthorized access, data exfiltration, data poisoning, model theft, code injection, unpatched dependencies, weak monitoring, and limited security awareness. Their recommendations include managed identities, strict access control with full logging, network isolation, end-to-end encryption, comprehensive input and output validation, provenance tracking, anomaly detection, adversarial training, careful dependency management, and incident response planning. Framing these practices as part of the MLSecOps discipline, their work emphasizes the need for stronger protection of data and models together with continuous monitoring and incident readiness. However, their catalog does not explicitly map attacker tactics to lifecycle stages or to public knowledge bases such as

ATLAS. Our survey builds on their foundation by using ATLAS to organize risks and defenses according to specific tactics and techniques observed in real MLOps deployments.

Ahmad et al. [3] extend the discussion to operational technology environments and argue that traditional IT controls are insufficient where machine learning systems impact physical processes and safety-critical applications. They call for comprehensive risk management across data, models, pipelines, infrastructure, personnel, and third-party dependencies, supported by data encryption, rigorous access control, anonymization, secure model versioning and storage, secured CI/CD pipelines, automated code scanning, container-based deployment, and continuous monitoring with audits. Zhang et al. [257] introduce the SecMLOps paradigm, embedding security throughout the lifecycle using a People, Processes, Technology, Governance, and Compliance lens. They define clear security responsibilities for stakeholders including business leads, data engineers, data scientists, DevOps engineers, and MLOps engineers. They also organize security activities such as threat modeling, policy definition, model security evaluation, and incident response into design, experimentation, and production stages, while considering how these intersect with fairness, explainability, reliability, safety, and sustainability.

Panchumarthi [219] proposes a DevSecMLOps framework that adapts DevSecOps practices for machine learning by covering data ingestion with automated detection and anonymization, model training with dependency scanning and reproducibility checks, model packaging with container scanning and software bill of materials generation, deployment using Zero Trust controls and policy as code, and operations with drift detection and automated incident response. Case studies in that work show improvements in vulnerability detection, lower false positive rates, better policy compliance, and faster drift detection. The framework also generates audit-ready artifacts to support regulatory requirements including GDPR, HIPAA, SOC 2, ISO 27001, and PCI DSS. While these works outline key elements of secure MLOps and SecMLOps, they do not provide a comprehensive mapping between attacker tactics, lifecycle stages, and operator-facing responsibilities across the ecosystem.

Our survey addresses these gaps by taking a lifecycle-oriented and operations-focused view that treats MLOps and LLMOps as interconnected systems rather than isolated components. We use ATLAS to connect attacker tactics to specific decisions and components across the lifecycle, interpreting these connections through two main perspectives: one for researchers who study attacks and defenses, and one for security and operations teams who deploy and maintain these systems as shown in Fig. 2. From the research perspective, our mapping offers a structured way to relate attack families such as poisoning, privacy, and evasion to the broader MLOps context and to highlight where existing techniques provide coverage and where gaps remain. From the operational perspective, our taxonomy tracks how design, platform configuration, development, deployment, and monitoring strategies influence exposure to specific ATLAS tactics. This dual perspective positions our survey relative to prior work and focuses attention on lifecycle stages that are most critical for operators and end users. The next subsection formalizes the taxonomy that underpins this approach and provides the structure used in later sections to analyze attacks, defenses, and attacker capabilities. By grounding this taxonomy in both the literature and operational practice, we aim to make it useful for both system designers and those responsible for running systems in real-world environments.

3.2 Survey Taxonomy

Guided by the landscape above, the MLOps and LLMOps ecosystem can be grouped into three families where security-relevant assets and attack surfaces are most concentrated: *Design and Setup*, *Model Development and Evaluation*, and *Operations*. Instead of restating the full lifecycle, these families are treated as clusters of components and interfaces that are accessible to attackers and serve as key points for organizations to apply controls, defenses, and mitigations. Fig. 3 illustrates this taxonomy and shows how attack paths can cross family boundaries.

3.2.1 Design and Setup. *Design and Setup* forms the control plane where organizations define resource access, environment structure, and trust relationships with external services. Core assets include identity and access

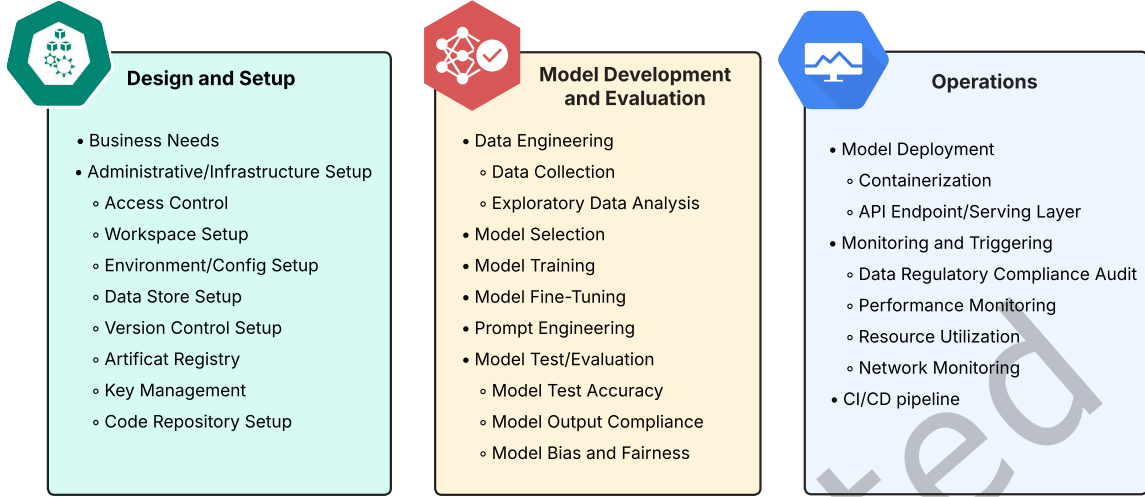


Fig. 3. An overview of our proposed taxonomy. The MLOps lifecycle is structured into three distinct and non-overlapping families: Design and Setup, Model Development and Evaluation, and Operations. The bullet points listed under each family specify the corresponding phases within the lifecycle. This organizational structure is adapted from the principles in ml-ops [244] and is equally applicable to LLMOps.

control systems, workspace and project layouts, configuration and secrets management, infrastructure as code, version control systems, and artifact registries. For LLMOps, this also covers vector stores, prompt templates, safety policies, and tool connectors used in later stages. The main attack surface involves privileged accounts, configuration channels, and shared infrastructure. Attackers may target identity providers and policies to gain or escalate access, exploit misconfigured isolation to cross project boundaries, tamper with infrastructure as code and base images, or modify repositories, registries, and secrets stores. These design decisions directly influence which attacks become feasible in other families and determine which defenses and mitigations remain effective throughout the lifecycle.

3.2.2 Model Development and Evaluation. *Model Development and Evaluation* includes the assets and workflows that turn raw or curated data into trained models, prompts, and evaluation artifacts. Key components are data pipelines, labeling workflows, notebooks and integrated development environments, experiment tracking, training clusters, job schedulers, and model or prompt registries. For LLMOps, this extends to prompt design environments, libraries, and evaluation tools. The main attack surface is shaped by data flows and development interfaces. Data ingestion can be exploited for poisoning, labeling tools and human-in-the-loop processes may be used to bias labels or steal information, and notebooks and trackers often operate with broad permissions that allow data theft or backdoor insertion. Training jobs and pipelines may be misconfigured to swap datasets or reroute outputs, while model or prompt registries may be altered so that compromised artifacts are mistakenly approved for deployment.

3.2.3 Operations. *Operations* covers the runtime surfaces where deployed models and LLM configurations interact with users, external systems, and monitoring tools. Assets in this family include serving layers and API endpoints, inference jobs, online feature and context stores, monitoring and logging systems, safety and policy engines, routing and orchestration components, and CI/CD pipelines. In LLMOps, this also covers vector search services, tool execution frameworks, and gateways that proxy calls to external providers. Public or partner-facing

endpoints can be exploited to send malicious inputs for prompt injection, model theft, or resource exhaustion. Weak authentication or opaque routing may lead to tenant data exposure. Feature stores, context stores, and vector indices can be poisoned or queried for sensitive data, while monitoring and logging systems may be manipulated to conceal or exaggerate incidents. CI/CD pipelines present a tightly connected surface where modified artifacts, policies, or configurations can be introduced as legitimate updates.

Across all three families, the taxonomy separates attack surfaces according to dominant assets and the specific interactions each surface exposes to attackers. *Design and Setup* focuses on persistent control plane decisions, *Model Development and Evaluation* centers on data and training workflows, and *Operations* highlights runtime exposure and automated change. This structure is used in Section 3.3 to formalize the threat model and to organize the discussion of attacker techniques, defenses, and mitigations in later sections.

3.3 Threat Model and Capabilities

Building on the taxonomy and incorporating ATLAS [208] and related references, this subsection develops a threat model from the perspective of the attacker, illustrating how adversaries develop motivation, gather knowledge, and acquire the capabilities necessary to compromise MLOps and LLMOps environments. The model conceptualizes the attacker’s progression through three interconnected phases: understanding the reasons for targeting machine learning systems (motivation and impact), identifying the assets and surfaces to target (intelligence and knowledge), and developing the means to execute attacks (capabilities and staging). These phases yield attacker profiles ranging from external black-box adversaries to privileged insiders, each engaging with the principal attack surfaces defined across the MLOps lifecycle.

3.3.1 Motivation and Impact. From an adversarial standpoint, MLOps and LLMOps environments present attractive targets because they control critical decisions and often lack comprehensive security controls [144]. Attackers seek to influence the integrity, availability, confidentiality, and operational cost of machine learning systems, as well as to undermine the perceived trustworthiness of these platforms. Their motivations may be immediate and tactical, such as evading detection or reducing system reliability, or more strategic, involving persistent access and long-term manipulation. For example, attackers focused on short-term objectives may seek to bypass machine learning-based security controls by crafting adversarial inputs that cause misclassification [98, 128]. Adversaries may also deny machine learning services through resource exhaustion attacks or by introducing chaff data, which degrades trust and may force organizations to revert to manual processes [100, 125].

Economic drivers include cost abuse and intellectual property theft, where adversaries manipulate cloud-based machine learning workloads to increase operational expenses for victims or exfiltrate proprietary models and datasets to accelerate their own development [97, 142]. Privacy-focused attackers exploit the capacity of models to memorize and leak sensitive or confidential information, leading to legal liability and reputational harm [143, 144]. Advanced attackers may pursue persistent objectives by poisoning training data or corrupting models, introducing hidden failure modes that persist beyond initial access [104, 173]. Documented incidents, including backdoor attacks with neural payloads, identity verification bypasses, and sensor-based adversarial manipulations, exemplify the diversity and severity of these threats [39, 67, 141, 144, 221]. Understanding these motivations is a prerequisite for analyzing how attackers gather intelligence and build the knowledge required to mount effective attacks, which is examined next.

3.3.2 Intelligence and Knowledge. Attackers conduct intelligence gathering through processes that typically progress from black-box reconnaissance to grey-box discovery and, eventually, to white-box collection. During black-box reconnaissance, adversaries collect passive intelligence by analyzing conference papers, technical blogs, and public vulnerability reports to understand deployment patterns and architectural choices [120–123, 180, 181]. Active network scanning enables the identification of publicly exposed endpoints and configuration

weaknesses, while investigation of application stores and victim websites reveals available machine learning APIs [119, 124, 129]. In this phase, attackers interact only through public interfaces, inferring model properties from system outputs and timing behaviors, and launching attacks such as model extraction or model stealing [78] despite incomplete knowledge.

With partial internal access, the threat escalates to the grey-box phase. Here, attackers leverage compromised credentials or leaked documentation to map the landscape of machine learning artifacts, including software stacks, feature stores, and deployment pipelines [101, 138]. They interrogate model behavior to deduce model family and output space, and tailor their attack strategies accordingly [102, 103]. In the context of LLMs, grey-box adversaries extract meta-prompts, probe for hallucinations, and analyze model confidence scores to refine their attacks [147, 171, 172]. Demonstrations by red teams, such as those conducted by MITRE, have illustrated that partial internal knowledge can be sufficient to design effective adversarial inputs targeting model weaknesses [88]. The ability to align exploitation techniques with specific frameworks further increases the reliability and precision of attacks [180].

In the white-box phase, adversaries obtain comprehensive access, often through insider roles or compromised administrative credentials [132]. Attackers in this position extract models, datasets, and telemetry from internal repositories, thus gaining visibility into data distributions, training procedures, and hyperparameter choices essential for sophisticated and persistent attacks [109, 132]. Collaborative platforms and local system files provide additional entry points for lateral movement and further data exfiltration [135, 136]. White-box access allows for precise computation of adversarial perturbations and targeted data extraction. Incidents such as the Clearview AI breach and disruptive red team exercises targeting cloud machine learning platforms highlight the risks associated with this level of compromise [89, 253]. The depth of attacker knowledge directly influences both the feasibility and reliability of the techniques that can be deployed [132, 138, 180]. Once attackers acquire the necessary intelligence, the next critical step is to translate this knowledge and motivation into operational capabilities, as discussed below.

3.3.3 Capabilities and Staging. Adversaries develop attack capabilities by acquiring technical knowledge from academic literature and online resources, and by utilizing open-source frameworks such as CleverHans, ART, and Foolbox, which automate adversarial example generation and lower the barrier for executing advanced attacks [36, 55, 223]. These frameworks are often combined with general-purpose scripting tools, established machine learning libraries such as PyTorch and TensorFlow, and command-line interpreters that support the automation of exploitation activities [51, 133, 153].

To build attack infrastructure, adversaries rent servers, domains, and scalable cloud resources for staging and execution, often selecting GPU-enabled environments provided by Google Colab, AWS, and Azure to support computationally intensive workloads [16, 82, 86, 91, 92]. Activities are distributed across multiple accounts and domains to obscure attack traffic and hinder detection. In some cases, attackers exploit virtualization and cloud management vulnerabilities to escalate privileges or pivot to additional targets within the environment [218].

The collection of machine learning artifacts from public repositories represents another critical capability. Adversaries identify datasets and pre-trained models that closely resemble the target environment, using these proxies to develop transferable attack techniques [93–95]. The establishment of seemingly legitimate accounts provides a cover for extended reconnaissance and the subsequent staging of attacks [139].

Supply chain compromise is an increasingly prevalent vector for persistent access. Attackers inject malicious packages or dependencies into the software ecosystem, leveraging techniques such as dependency confusion to insert backdoored components through naming collisions in package management systems. The PyTorch dependency confusion incident, which resulted in exfiltration of developer data, exemplifies this threat [13, 114, 154]. Advanced adversaries may also craft custom attack tools, including adversarial websites, notebook templates

embedded with exfiltration mechanisms, or scripts designed to exploit collaborative platforms. Scripting languages such as Python, PowerShell, and Unix shells are common vectors in these workflows [50, 51, 83, 133, 218].

The final operational phase centers on staging and delivering the attack. Adversaries publish poisoned datasets and models in public or private repositories, or manipulate training data within established pipelines to introduce backdoor triggers and vulnerabilities [117, 118]. Compromised models are uploaded to public registries in the expectation that downstream consumers will adopt them, often during fine-tuning or deployment processes [96, 179]. In LLM-focused environments, attackers fabricate hallucinated entities, such as fictitious packages or domains, to mislead users and security teams. Infrastructure is established with persistent accounts and domains, ensuring continued access even as legitimate users update or replace poisoned artifacts [96, 117, 155, 167]. Taken together, these capabilities enable attackers to compromise MLOps environments by progressing from motivation and intelligence gathering to the acquisition and staging of operational tools. The following section examines concrete attacks on MLOps and LLMOps systems, and presents defenses and mitigations organized according to these attacker profiles and lifecycle stages.

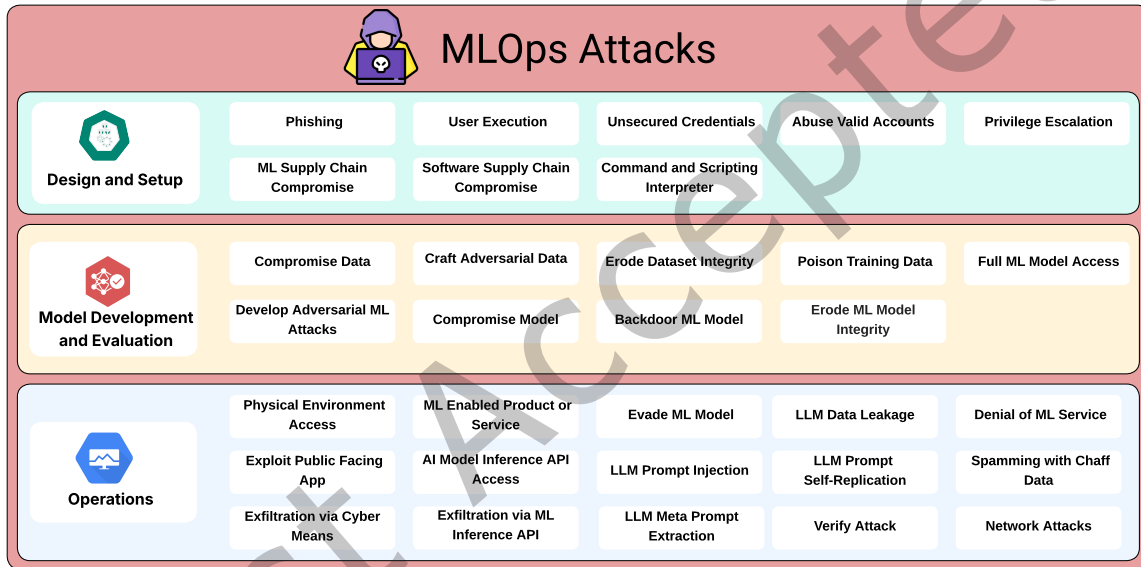


Fig. 4. Attacks are organized according to the taxonomic categories presented in Section 3.2. The three groups act as analytical lenses rather than literal or compressed lifecycle phases and support a structured comparison of how diverse threats concentrate around design choices, model-centric workflows, and operational deployment contexts.

4 Attack and Challenges Review

Building on the survey approach and threat model described earlier, this section turns to concrete attacks that target the MLOps and LLMOps ecosystems in order to address RQ2. Our analysis is organized around three taxonomical categories introduced in Section 3.2: *Design and Setup*, *Model Development and Evaluation*, and *Operations*, which are depicted in Fig. 4. For each category, we map relevant attack techniques from the MITRE ATLAS framework to real-world incidents and red-teaming exercises, distinguishing between threats that have materialized in practice and those that remain theoretical. This evidence-based approach aims to give practitioners actionable intelligence about the threat landscape and inform both organizational and technical risk management

decisions. Across all three categories, attack paths frequently manifest as supply chain attacks that target users or systems, which highlights the interconnected and layered nature of vulnerabilities present throughout the MLOps lifecycle.

4.1 Attacks in Design and Setup

The *Design and Setup* phase is where organizational and technical choices establish the foundation for the MLOps environment. Attacks at this stage often exploit weaknesses in human factors, identity management, configuration, and core infrastructure to gain initial access to machine learning systems. Human users are frequently regarded as the weakest link in security [232], and attackers exploit this reality by combining social engineering with technical manipulation. Their goal is to escalate from basic access to more advanced forms of exploitation. Initial access is commonly gained through phishing, social engineering, user execution, and privilege escalation, prompting users to take unintentional actions that activate malicious code or introduce unsafe packages, which ultimately undermines operational integrity.

Phishing remains one of the most effective entry points into MLOps environments [156, 200]. Traditional phishing relies on generic fraudulent communications, while spearphishing targets individuals with highly personalized messages [157]. The evolution of these attacks is evident in recent incidents where generative AI and deepfake technologies are used to craft sophisticated, context-aware messages that closely mimic legitimate organizational communication [26, 157]. The Computronix AI Phishing Campaign highlights this shift. In this incident, attackers leveraged automation and natural language generation to create highly personalized messages that bypass spam filters and deceive even vigilant, security-conscious users [26]. These AI-powered campaigns allow attackers to scale their operations efficiently, preserving context-specific targeting and achieving higher success rates than traditional phishing attempts. In parallel, user execution attacks [163] manipulate users into executing malicious code, often introduced through compromised components in the machine learning supply chain [111]. Unsafe ML artifacts can carry harmful payloads, while serialization vulnerabilities in model storage or loading routines may be exploited for code execution if proper validation is lacking [126, 218].

An illustrative case is the PyTorch Dependency Confusion attack, where attackers uploaded a malicious package to the public PyPI repository. This package impersonated a legitimate PyTorch dependency and, during routine updates, was automatically installed by many users. As a result, sensitive system information and SSH keys were exfiltrated to attacker-controlled servers [13, 163, 183]. The incident highlights how trust in software package ecosystems can be weaponized through what appear to be routine update procedures. Similarly, the ChatGPT Package Hallucination incident exploited hallucinations in LLM outputs by registering non-existent packages that users, relying on LLM recommendations, subsequently installed. This led to compromise of their systems [106].

Privilege escalation attacks represent another powerful tool for attackers, enabling them to expand access and achieve broader objectives within the MLOps environment [158]. Such escalation may result from LLM prompt injection [150, 151], LLM plugin compromise [148], or jailbreak techniques [146]. The ChatGPT Plugin Privacy incident demonstrated how excessive plugin permissions can become a critical vulnerability. In this scenario, attackers used markdown injection to embed malicious links that exfiltrated sensitive conversation data from users [158, 250]. The Morris II worm highlighted the risk of indirect prompt injection, where retrieval-augmented generation in an email assistant enabled self-replication and automated exfiltration of sensitive data across multiple systems [254]. These examples demonstrate the necessity for strong privilege boundaries and rigorous access controls.

Attacks targeting access abuse naturally follow, as attackers take advantage of legitimate credentials or authentication tokens. These include API keys, service tokens, and cloud account credentials, which allow them to bypass conventional security controls and manipulate machine learning infrastructure [186]. Weakly managed

credentials such as default accounts, dormant accounts, and cloud credentials present attractive opportunities [165, 204–207]. During a red-teaming exercise on Microsoft Azure ML services, attackers used compromised credentials to escalate from basic user privileges to administrative access. They ultimately corrupted model outputs and exfiltrated sensitive training data [89]. Similarly, a Google Colab vulnerability enabled arbitrary code execution, which allowed attackers to infiltrate internal services and escalate privileges through exposed cloud credentials [1].

Unsecured credential exposure further amplifies these risks, especially when secrets are stored in plaintext, in source code, or as environment variables across cloud and containerized environments [182, 202]. The MathGPT incident illustrates this challenge: exposed API keys and authentication tokens enabled attackers to access high-value resources and sensitive training data, causing significant financial and security damage [53]. Attackers also harvest secrets by exploiting access to cloud instance metadata APIs, Kubernetes clusters, Docker images, and even collaboration tools such as Slack and Trello [11, 19, 76, 203, 237]. Automated tools and scripts have made credential harvesting highly efficient, and the Ray framework vulnerability showed how widespread the consequences can be when thousands of deployments are exposed at scale [71]. The difficulty in distinguishing malicious from legitimate use is exacerbated when credential hygiene is poor and monitoring is insufficient.

Beyond these risks, infrastructure attacks focus on the critical systems that support machine learning workflows. These include servers, CI/CD pipelines, GPUs, version control systems, and collaborative environments [23]. Compromising any of these components allows attackers to manipulate or delay model training, testing, or deployment. Many attacks involve the insertion of malicious code through scripting languages commonly used in MLOps, such as Python and PowerShell [51, 83, 133]. Because these scripts underpin core CI/CD workflows, attackers can disguise their activities within routine operations, making detection challenging in environments that depend on continuous integration and deployment [218]. Hardware compromise and malicious software supply chain attacks can introduce adversarial examples or backdoored libraries that persist throughout the ML lifecycle [113, 114, 154]. Together, these attacks reveal how initial setup and configuration decisions can propagate vulnerabilities that undermine the integrity of machine learning operations. The next subsection examines attacks that target the core processes of model development and evaluation.

4.2 Attacks in Model Development and Evaluation

The *Model Development and Evaluation* phase faces attacks that compromise data integrity, model behavior, and the security of training processes. Unlike threats in the *Design and Setup* phase that primarily aim to obtain access, adversarial techniques here focus on corrupting the machine learning artifacts themselves. These artifacts include training data, fine-tuning data, labels, model architectures, weight parameters, and pre-trained components. Such attacks introduce subtle and persistent vulnerabilities that may remain undetected during evaluation and continue to influence system behavior throughout deployment and production use. Because modern ML development depends on large-scale automated data pipelines, public datasets, and the reuse of shared model components, this phase becomes a critical point where attackers can shape system behavior at scale and with long-lasting effect.

A central adversarial objective is the degradation of data integrity, since both training and operational data must remain accurate, representative, and trustworthy. Poisoning attacks embed harmful samples or misleading content into pre-training or fine-tuning datasets, thereby degrading model performance or embedding hidden behaviors that activate under specific conditions [117, 218]. Early incidents such as the Microsoft Tay Chatbot failure demonstrated the risks of unfiltered continuous learning, where coordinated adversaries exploited the system's exposure to public social media interactions and triggered rapid degradation of model behavior [217]. More advanced poisoning strategies such as split-view poisoning exploit predictable data collection schedules at web scale. Attackers inject malicious content into webpages shortly before automated crawlers run, ensuring poisoned data enters the training pipeline even though ordinary users observe a different version of the webpage [22].

Experiments have shown that poisoning as little as 0.01 percent of a dataset can produce targeted misclassifications while headline accuracy metrics remain stable, making detection through standard validation extremely difficult [173].

Beyond direct poisoning, training-only manipulation attacks such as feature collision, label flipping, and bilevel optimization introduce subtle vulnerabilities that bypass conventional detection [43]. Federated learning systems amplify these risks because poisoned weights can be injected into the aggregation process from multiple decentralized nodes, complicating attribution and undermining global model robustness. Attackers may also insert false labels, malware samples, or mislabeled content into open-source datasets, causing organizations to unknowingly import tainted samples into their pipelines [118, 251]. These threats extend into the inference stage, where adversarial inputs crafted under black-box and white-box conditions induce misclassification without modifying training data. Embedded backdoor triggers enable additional targeted manipulation by activating hidden behaviors only when specific patterns or tokens appear in inputs [99].

Real-world incidents illustrate how such adversarial techniques accumulate over time to degrade entire security ecosystems. The VirusTotal poisoning campaign revealed how systematically submitting borderline malware samples could shift decision boundaries for multiple commercial detection models, eventually enabling new malware variants to evade detection across the industry [90]. Similarly, adversarial capabilities demonstrated by Kaspersky researchers showed how perturbations crafted against local feature extractors could transfer successfully to cloud-based antimalware systems, proving that attackers can develop and test attacks locally before deploying them at scale against remote services [4]. These insights demonstrate that adversaries frequently refine their attack strategies through the use of open-source adversarial ML frameworks. Toolkits such as CleverHans, ART, and Foolbox automate adversarial example generation, backdoor construction, and robustness evaluation, lowering the barrier for developing advanced attacks and enabling even relatively low-resourced adversaries to reproduce sophisticated techniques [36, 55, 223].

While data integrity attacks focus on corrupting training inputs, model integrity attacks target the internal architecture, learned representations, and distribution mechanisms of ML models. These attacks degrade decision-making capability, create opportunities for the extraction of private information, and enable the covert spread of misinformation. Backdoor attacks embed malicious triggers into models through poisoned data or direct parameter manipulation, causing the system to behave normally under standard evaluation yet reveal attacker-controlled behavior when specific inputs appear [96]. Hidden triggers undermine reliability and force organizations to expend significant resources investigating and repairing corrupted models [104]. When adversaries possess full access to model architectures and parameters, they can design highly precise backdoors or payloads customized for particular tasks, users, or operational environments [108].

Deep payload injection represents one of the most concerning manifestations of model integrity compromise. In this attack, executable code is embedded directly into neural network weight matrices, converting a seemingly benign ML model into a covert malware delivery mechanism [67]. Researchers identified dozens of vulnerable Android applications whose on-device models performed tasks such as cash recognition, user authentication, parental control, and financial processing. Although the corrupted models produced normal inference outputs, they enabled data exfiltration, command execution, and unauthorized behavioral modification. Detection through functional testing became practically impossible because the underlying prediction behavior remained intact.

Model supply chain compromise magnifies these risks by targeting pre-trained and publicly shared models. Threat actors have uploaded malicious or backdoored models to widely used repositories such as Hugging Face, where unsuspecting developers may integrate them directly into downstream systems without realizing that dormant triggers or embedded payloads remain hidden within the model [179]. The PoisonGPT demonstration exemplified this risk by releasing a modified large language model that passed standard benchmark evaluations but returned targeted misinformation when queried about specific topics [28]. Because such attacks exploit the trust placed in public model hubs and the widespread reuse of pre-trained model components, compromised

artifacts can propagate across organizations and industries. Outdated or poorly maintained models further increase exposure because they often lack important security updates and remain vulnerable to parameter editing techniques, including emerging methods like Rank-One Model Editing (ROME) [218].

Taken together, these attacks show how adversaries can quietly and systematically undermine both the data foundations and the computational logic of ML systems during development and evaluation. By corrupting datasets, manipulating learned representations, and compromising pre-trained artifacts distributed across shared ecosystems, attackers influence downstream predictions long after deployment. The cumulative effect is an erosion of trust in model reliability, increased vulnerability to operational exploitation, and long-term degradation of system security. Having examined these foundational weaknesses in model development and evaluation, the next subsection turns to attacks in the Operations phase, where adversaries exploit compromised models and the surrounding infrastructure during deployment and real-time interaction.

4.3 Attacks in Operations

The *Operations* phase exposes deployed machine learning and large language model systems to direct attacker interaction through public APIs, user interfaces, and network endpoints. In this stage, attackers leverage runtime vulnerabilities to extract information, evade detection, or disrupt mission-critical services. Operational attacks often combine techniques across physical, application, and infrastructure layers, with these actions unfolding in real time and leading to significant consequences. As machine learning and large language model deployments continue to grow in various domains, the scale and severity of operational risks have become more pronounced, making it critical to examine how these attacks manifest within modern system architectures [169, 171].

Physical manipulation of sensor inputs represents a significant operational risk, especially for systems that process real-world data. Bypassing biometric authentication is possible when physical security is circumvented using engineered artifacts. For example, red team researchers have achieved targeted misidentification in facial recognition systems using printed masks and infrared LEDs, resulting in both impersonation, where unauthorized users gain access, and anonymization, where legitimate users are not recognized [88, 116]. These events demonstrate that attackers can compromise machine learning systems using physical means alone, without the need for digital intrusion. Identity verification platforms are also susceptible to attacks that exploit synthetic media such as deepfakes. In the Bypassing ID.me incident, adversaries were able to defeat liveness detection and identity matching mechanisms, which allowed them to fraudulently claim government benefits and convert these weaknesses into financial gain [39, 110].

Attacks on service-level and infrastructure components target the core platforms and tools that support machine learning workloads. Publicly accessible APIs, as shown by the ShadowRay exploitation campaign, provide attackers with opportunities to exploit unauthenticated endpoints in cloud-based deployments. Adversaries in this case executed arbitrary code, deployed cryptocurrency miners, exfiltrated sensitive training data, and established persistent access by targeting Ray Jobs APIs, which led to the compromise of more than five thousand servers and significant financial and operational losses [71, 140]. These attacks are not limited to resource theft. Attackers have also initiated resource exhaustion campaigns, as demonstrated in the MathGPT incident, where a lack of rate limiting allowed automated large-scale API queries to overwhelm computational resources and increase operational costs [53, 100]. Collaborative environments such as Google Colab are at risk for code execution attacks, in which adversaries inject malicious scripts into shared notebooks. When these notebooks are opened, attackers can exfiltrate datasets, credentials, and proprietary code, resulting in the compromise of research efforts [1, 107].

A critical area of operational attacks centers on the confidentiality and privacy of training data, model internals, and user information. Attackers have developed techniques to infer whether certain data samples were included in training sets and to extract global statistical properties or reconstruct portions of the original data. These

methods often use shadow models or detailed analysis of prediction scores [106]. Such data exfiltration through inference APIs introduces substantial privacy concerns, as models may reveal personally identifiable information or intellectual property present in training data. Large language models are particularly vulnerable to data leakage attacks in which specific prompts elicit sensitive or proprietary information, including material from proprietary corpora, public sources, or even previous user sessions. These vulnerabilities have become major security concerns for interactive machine learning and large language model deployments, since models with high capacity may unintentionally reveal information that affects organizational trust and regulatory standing.

Operational risks also include model extraction attacks, where adversaries systematically query inference APIs to replicate proprietary models or steal intellectual property. In the Machine Learning Translation attack, researchers used strategically designed API queries to clone commercial translation systems, effectively recreating models that required years of investment [169, 246]. Evasion attacks further challenge operational security by introducing adversarial inputs that can bypass detection. Studies of domain generation algorithm detectors and AI-powered antivirus tools such as Cylance have shown that small character-level or byte sequence modifications can defeat these models while maintaining malicious intent [127, 128, 236, 252]. Attackers typically validate such methods in controlled or low-risk environments before targeting production systems, allowing them to refine attack strategies and reduce the chance of early discovery [127].

Large language models expand the operational attack surface because they rely on natural language interfaces and often integrate with third-party tools. Prompt injection attacks use conversational context or document uploads to insert hidden instructions, which may cause models to leak sensitive information, produce misleading outputs, or circumvent safety measures [149, 235]. The Morris II worm incident is an example of a self-replicating attack in large language model applications. Here, malicious prompts autonomously spread by extracting and forwarding context to new recipients, enabling the infection to propagate across different organizations [145, 177, 254]. Meta prompt extraction attacks allow adversaries to discover hidden system instructions or internal logic, which can expose proprietary knowledge or change how the model behaves [147, 218]. Incidents such as these demonstrate that products and services powered by machine learning are frequent targets, and attackers continue to search for operational entry points and develop attack methods specific to the machine learning lifecycle [110].

Network exploitation in the operational phase is another critical threat, where attackers target interconnected infrastructure to disrupt services, compromise data, or degrade pipelines [100, 229]. By taking advantage of exposed endpoints and weak communication protocols, attackers can manipulate workflows, exfiltrate information, inject poisoned or malformed data, or make services unavailable for legitimate users [107, 140, 229]. Notable incidents involving network-based attacks, including those targeting Ray Jobs APIs or overloading systems with chaff data, reveal how vulnerabilities in a single area can cascade and put entire platforms at risk. Attackers may also gain unauthorized access by hijacking sessions, forging credentials, or manipulating network configurations. In distributed machine learning environments, proactive measures such as robust network segmentation, ongoing traffic monitoring, anomaly detection, and strong authentication protocols are needed to mitigate these risks [190, 193, 194, 197, 198].

Taken together, these operational attacks show that machine learning and large language model systems are exposed to continuous and evolving threats after deployment. The scope of attacks includes physical sensor manipulation, API abuse, code execution, data and model exfiltration, adversarial querying, prompt injection, meta prompt extraction, and complex network exploitation. When analyzed alongside attacks in the Design and Setup as well as Model Development and Evaluation phases, which focus on human factors, data pipelines, and model artifacts, the operational phase reveals the entire spectrum of layered risks present in today's MLOps and LLMOps environments. Attackers rely on both conventional information technology techniques, such as phishing and credential theft, and machine learning-specific strategies, including poisoning, backdoors, model extraction, adversarial querying, and prompt injection. Real-world incidents and red teaming exercises have

demonstrated that these threats are not only theoretical but have resulted in measurable operational, financial, and reputational damage. Table 1 presents a synthesis of attack techniques observed across the machine learning lifecycle, their empirical validation, and how they overlap with traditional information technology security concerns. The following section transitions from the analysis of attacks to an in-depth discussion of defenses and mitigation strategies designed to strengthen MLOps and LLMOps deployments against persistent adversarial threats.

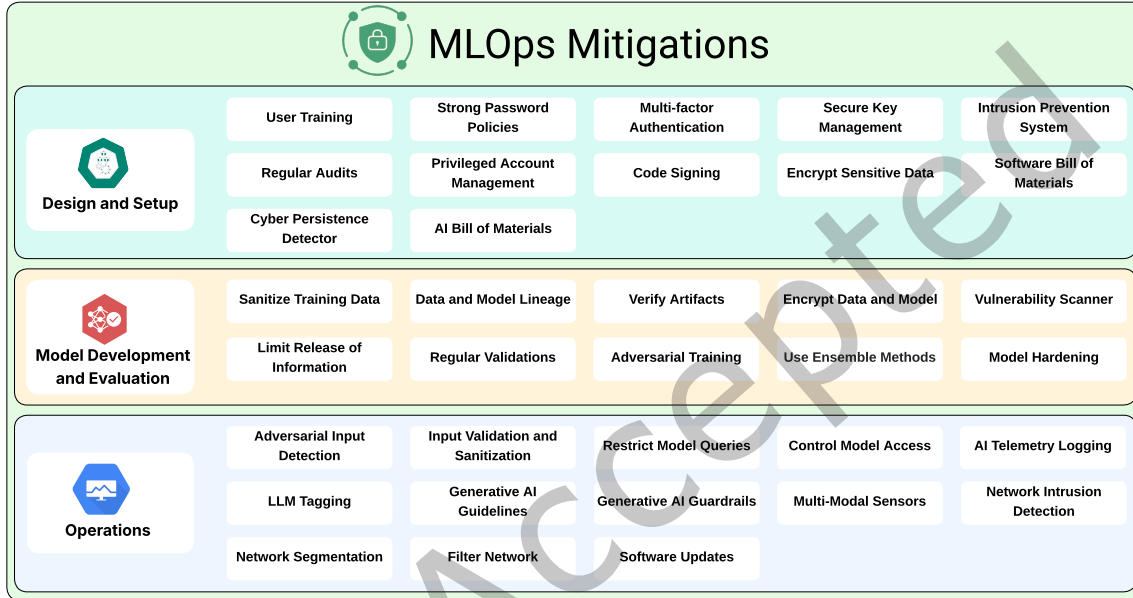


Fig. 5. Mitigation strategies organized according to the taxonomic categories presented in Section 3.2, corresponding to the attacks shown previously. These categories enable systematic alignment of defenses with design choices, model-centric workflows, and operational deployment contexts.

5 Security Mitigation Strategies

The previous section outlined the attack landscape and identified key vectors that attackers may exploit across the MLOps lifecycle. This section turns to corresponding mitigation strategies. To address RQ3, we assess how well the mitigation techniques proposed in existing frameworks align with the identified attack techniques and whether they provide comprehensive coverage of the resulting attack surface. Drawing from MITRE ATLAS, MITRE ATT&CK, and recent research, we group defenses by the same three categories used in the attack taxonomy. Fig. 5 summarizes the mitigation techniques associated with each phase of the lifecycle and serves as a visual reference for the discussion that follows. For each phase we highlight concrete mitigation techniques, relate them to real world deployments, and explain how they jointly constrain attacker opportunities introduced in Section 4. We begin by examining mitigations for the Design and Setup phase, which establish the security foundation for the entire MLOps pipeline.

Table 1. Summary of MLOps attack techniques with empirical validation. Traditional IT column indicates attacks that overlap with conventional cybersecurity (✓) or are unique to ML environments (×). The empirical validation column cites real world incidents or red team exercises that demonstrate practical exploitation of each technique.

Taxonomy	Attack Techniques	Empirical Validation	Traditional IT
Design and Setup	Phishing [156]	AI Phishing Campaign [26]	✓
	Unsecured Credentials [182]	MathGPT incident [53]	✓
	Privilege Escalation [158]	ChatGPT Plugin Privacy [250]	✓
	User Execution [163]	PyTorch Dependency [13]	✓
	Abuse Valid Accounts [165]	Azure Service Disruption [89]	✓
	ML Supply Chain Compromise	HuggingFace Malicious Model [255]	✓
Model Development and Evaluation	Compromise Data [112]	Microsoft Tay Chatbot [217]	×
	Erode Dataset Integrity [173]	Split view poisoning [22]	×
	Poison Training Data [117]	Split view poisoning [22]	×
	Compromise Model [115]	Deep Payload Injection [67]	×
	Develop Adversarial ML Attacks [137]	Kaspersky Adversarial Attack [4]	×
	Craft Adversarial Data [98]	VirusTotal poisoning [90]	×
	Backdoor ML Model [96]	PoisonGPT [28]	×
	Erode ML Model Integrity [104]	VirusTotal poisoning [90]	×
	Full Model Access	Deep Payload Injection [67]	×
Operations	Physical Environment Access [116]	Face Identification System [88]	×
	Exploit Public Facing Application [140]	ShadowRay Exploitation [71]	✓
	ML Enabled Product or Service [110]	Bypassing ID.me [39]	×
	AI Model Inference API Access [169]	Machine Learning Translation [246]	×
	Verify Attack [127]	DGA Detection Evasion [252]	×
	Spamming with Chaff Data [125]	×	×
	Denial of ML Service [100]	MathGPT incident [53]	×
	Exfiltration via Cyber Means [107]	Google Colab Code Execution [1]	✓
	Exfiltration via ML Inference API [105]	×	×
	Evade ML Model [128]	Cyclance AI Malware Detection [236]	×
	LLM Prompt Injection [149]	AgentFlyer Copilot [235]	×
	LLM Meta Prompt Extraction [147]	×	×
	LLM Data Leakage [145]	Morris II worm [254]	×
	LLM Prompt Self-Replication	Morris II worm [254]	×

5.1 Mitigations for Design and Setup

Mitigations in the *Design and Setup* phase focus on preventing initial compromise and limiting the impact of successful entry into the environment. As indicated in Fig. 5, these defenses span user-focused training, administrative controls, and infrastructure-level safeguards that collectively shape the security posture of the MLOps and LLMOps stack. The remainder of this subsection explains how these classes of mitigation reduce opportunities available to attackers and how they have been applied in practice, progressing from user-facing controls through identity management to technical infrastructure protections and supply chain security.

User-facing mitigations address phishing, social engineering, and unsafe execution behavior through a combination of education and technical controls that work together to create multiple barriers against initial compromise. User training [164] forms the foundation of human-centered defenses, with security awareness training programs recommended to provide at least quarterly training sessions and simulated phishing tests to reduce organizational risk [60]. Organizations can further reduce phishing exposure and malicious code execution by deploying antivirus and antimalware tools, running regular audits and scans [195], and blocking suspicious traffic with intrusion prevention systems [185]. A study of 100 companies in Indonesia that implemented IT audits reported that 85% of organizations saw reduced risk of data breaches and 78% observed improved operational efficiency [233]. A classic Cisco case study shows how network-based intrusion prevention detected and mitigated internal security events before users experienced secondary impact [226]. These technical controls complement user education by providing automated defenses that can detect and block threats even when human vigilance fails, creating a defense-in-depth approach where multiple layers of protection work in concert to reduce the organization's attack surface.

Beyond user education and automated threat detection, strong identity and account controls serve as a critical barrier against credential theft and abuse of valid accounts. Strong password policies [192] establish baseline credential security, with one case study documenting a move from a six-character default to an eight-character minimum based on empirical work showing that eight-character passwords are much harder to break [215]. Multi-factor authentication [191] provides a more robust defense, with a large-scale benchmark of Microsoft Azure Active Directory (AD) users finding that over 99.99% of multi-factor authentication-enabled accounts remained secure and that this control reduces the risk of compromise by 99.22% across the population [77]. Privileged account management [201] further reduces risk by isolating administrative access from standard user workflows. A practical hardening step is to isolate all privileged accounts from standard user access and mandate multi-factor authentication for privileged access [234]. These identity controls work synergistically, with password policies providing baseline security, multi-factor authentication adding a second verification factor, and privileged account management ensuring that even compromised credentials have limited scope for damage through enforcement of least privilege, separation of duties, and behavioral analytics that help contain abuse when credentials are stolen, including temporal and sequence-aware models for detecting anomalous behavior over time [75].

Building on these identity controls, protecting credentials and configuration data also requires technical safeguards on stored secrets, system configuration, and data at rest and in transit. Encrypting sensitive information [187] reduces disclosure risks, with an enterprise encryption blueprint describing this strategy as protecting data in motion and at rest, with encryption keys as the primary asset to protect [240]. Secure key management using Hardware Security Modules and Trusted Platform Modules can further protect cryptographic keys [87], as seen in cloud key management services that support bring your own keys and hold your own keys models to keep customer keys under tighter control [46]. These encryption and key management practices must be complemented by access controls and system hardening to prevent unauthorized access to the underlying infrastructure. Restricting resource access over networks, enforcing restrictive operating system configurations, and restricting file and directory permissions narrow the opportunities for lateral movement once an attacker gains

initial access. Routine software updates reduce the number of exploitable vulnerabilities, behavior prevention on endpoints blocks malicious execution patterns, execution prevention stops unauthorized code from running, disabling unused features reduces the attack surface, and limiting software installation prevents the introduction of malicious or vulnerable components. Together, these technical safeguards create a hardened environment where credentials and sensitive data are protected through multiple layers of encryption, access control, and system configuration, while continuous monitoring and updating ensure that defenses remain effective against evolving threats.

The integrity of the MLOps environment depends not only on protecting existing infrastructure but also on ensuring that software and AI artifacts entering the system can be trusted, which requires supply chain-focused mitigations that verify the provenance and integrity of all components. A software Bill of Materials [34] catalogs all software components and dependencies involved in building systems, which enables rapid impact assessment when new vulnerabilities are disclosed. Recent large-scale incidents such as the SolarWinds and Log4j attacks highlight the importance of this visibility into the software supply chain. Code signing [131] is used to guarantee that software has not been altered, with a representative deployment centralizing signing profiles so that all code running on a serverless platform such as AWS Lambda must be signed by a trusted developer before execution [73]. For ML systems specifically, an AI Bill of Materials [168] extends this concept to track datasets, models, and training artifacts. In practice, the Neural Network Bill of Materials concept has been applied at scale to 55,997 GitHub repositories to move from conceptual AI Bills of Materials to concrete dependency tracking [227]. Verifying ML artifacts through cryptographic checksums mitigates risks of poisoned datasets or altered models, with a recent GPU-based framework named Sentry demonstrating practical cryptographic verification for ML artifacts, performing on-the-fly checks and achieving orders of magnitude speedup over CPU-based verification [40]. These supply chain mitigations provide critical assurance that both traditional software components and ML-specific artifacts can be traced to trusted sources and have not been tampered with during distribution or storage.

Finally, detecting long-lived persistence requires correlating events over time to identify attack patterns that span multiple stages of compromise and remain hidden within normal operational activity. Cyber Persistence Detector (CPD) [70] exemplifies this class of defenses. CPD links setup and execution events with pseudo-dependency edges, uses expert-defined edges and rules to flag suspicious chains of activity, and correlates related kill chain events into concise attack graphs. An alert budget system prioritizes high-value events for analyst review and focuses on behavioral patterns that are harder for attackers to evade than simple indicators. This type of persistence-focused monitoring is particularly important for MLOps environments where model and data assets live across many services and machines, creating numerous opportunities for attackers to establish persistent access that spans compute nodes, storage systems, and orchestration platforms. By correlating events across these distributed components, persistence detection systems can identify attacker footholds that would otherwise remain hidden within the noise of normal system operations. Taken together, these mitigations for the Design and Setup phase seek to reduce the probability of initial compromise through user training and technical controls, constrain the misuse of valid accounts through identity management, protect credentials and infrastructure through encryption and system hardening, preserve the integrity of software and AI components through supply chain verification, and detect persistent threats through correlation of events over time. Having established these foundational defenses, we now examine the mitigations that protect the model development and evaluation phase.

5.2 Mitigations for Model Development and Evaluation

Mitigations for the *Model Development and Evaluation* phase target the integrity and confidentiality of data, the robustness of training and validation pipelines, and the trustworthiness of model artifacts. Fig. 5 provides a

high level overview of the main defenses, and this subsection examines them in more detail by first considering data-focused mitigations, then detection and validation mechanisms, and finally model-focused measures. These complementary strategies work together to ensure that both the data feeding into models and the models themselves remain trustworthy throughout the development lifecycle.

Data-focused defenses aim to keep poisoned, redundant, or sensitive records out of training sets while preserving data utility for learning, requiring a careful balance between security and model performance. Sanitizing training data [160] identifies and removes compromised inputs before they enter the pipeline, using techniques such as anomaly detection, statistical outlier removal, and consistency checking to filter malicious samples. Data and model lineage [178] provides visibility into data origins and changes and enables comparison of current datasets with trusted baselines, creating an auditable trail that supports forensic analysis when poisoning is suspected. Broader work on adversarial ML mitigations emphasizes deduplication and sanitization to remove redundant or sensitive samples [35]. Encrypting data and models [35] secures training data, gradients, and model outputs from disclosure, with differential privacy injecting carefully calibrated noise to protect private information during training [256] and encryption-based approaches preventing unauthorized access to sensitive model parameters. Data anonymization and end-to-end encryption further protect personal information in training corpora [35]. To limit resource abuse during training, proof-of-work puzzles can slow extraction attempts [61], and profiling natural examples with energy consumption thresholds can help detect attacks that maximize resource usage to either degrade service availability or facilitate model extraction through repeated queries [57]. Limiting the release of information further constrains what attackers can learn about model internals, training data, or system architecture through both passive observation and active probing. Together, these data-focused mitigations create multiple barriers that protect training data from poisoning, preserve privacy through encryption and differential privacy, and limit the information available to attackers through careful control of what data and model details are exposed.

Complementing these preventive measures, detection and validation mechanisms provide ongoing assurance that data quality and model integrity are maintained throughout development, enabling teams to identify and respond to compromises before they propagate into production. Vulnerability scanning [184] can uncover weaknesses in model artifacts and in supporting software such as pickle file loaders, which are often used to serialize trained models but can execute arbitrary code when loading malicious files. An enhanced machine learning-based detection scheme for SCADA networks, for instance, achieved 99.49% accuracy, 99.23% precision, and 99.75% recall when identifying attacks [2]. Sanitizing training data remains important throughout the lifecycle, not just at the initial ingestion stage. In Anti Backdoor Learning, Li et al. [68] extend anomaly detection during training to remove both attack samples and non-regular traffic, leading to a more accurate detector with high detection rates and low false positives [68]. Regular validations [258] can reveal backdoors, adversarial bias, or data poisoning by testing for concept drift and shifts in training data distributions. One study proposed a mathematical framework for distribution shift and introduced a tangent space regularized estimator to control the shift and improve long-term accuracy [258]. Verifying artifacts [166] through cryptographic checksums ensures that datasets and models have not been tampered with during storage or transfer, providing a technical guarantee that complements behavioral detection methods. Maintaining a complete record of dataset sources and modifications supports these validation checks and improves transparency for later audits, enabling security teams to reconstruct the provenance of any suspicious artifacts and determine whether they originated from trusted sources or may have been compromised. These detection and validation mechanisms work in concert to provide continuous assurance, catching both subtle poisoning attempts that evade initial sanitization and supply chain attacks that introduce compromised artifacts at later stages of development.

While data quality and validation form the foundation, model-centric mitigations focus on robustness and integrity to ensure that the models themselves resist manipulation and perform reliably even under adversarial conditions. Adversarial training [72] strengthens models by exposing them to adversarial examples during the

training process, teaching them to maintain correct predictions even when inputs are perturbed. Recent Increasing Margin Adversarial training improves on prior methods by gradually enlarging classification margins and has been shown to outperform other defenses on six public image datasets for both classification and segmentation tasks [72]. Model hardening [152] encompasses a broader set of techniques including defensive distillation, gradient masking, and input preprocessing that strengthen models against manipulations or filter unusual inputs before inference [15]. Randomized smoothing and constraints such as Lipschitz continuity can yield certified robustness guarantees within defined perturbation thresholds [15], providing mathematical assurance that models will maintain correct behavior for inputs within a bounded distance from training examples. Detection tools build on anomaly detection or auxiliary models to flag suspicious samples, while explainable mitigation strategies help teams understand and respond to new attack patterns [15]. Using ensemble methods [161] can increase robustness against adversarial inputs by combining heterogeneous models that make errors on different samples, making it harder for attackers to find inputs that fool all models simultaneously. In vulnerability classification, a stacking learning classifier with heterogeneous ensemble methods has been shown to be more robust, reliable, and accurate than single models [247]. These model-centric mitigations create models that are inherently more resistant to adversarial manipulation, reducing the likelihood that attackers can exploit them through carefully crafted inputs or extract sensitive information through model inversion attacks. These mitigations for the Model Development and Evaluation phase therefore address data quality through sanitization and encryption, provide continuous validation through scanning and drift detection, and build robust models through adversarial training and ensemble methods. They aim to ensure that training data remains trustworthy, that models are evaluated under realistic adversarial conditions, and that all artifacts can be verified throughout the development lifecycle. With development phase protections in place, we next examine how these defenses extend into the Operations phase where models face real world threats.

5.3 Mitigations for Operations

Mitigations in the *Operations* phase address attacks that occur when models are deployed and exposed to users, services, and networks. Fig. 5 situates these defenses within the overall taxonomy, and this subsection discusses how they restrict access to deployed systems, provide telemetry for detection and response, harden inference against adversarial use, and address challenges specific to large language models. The operational security posture builds upon the foundations established in earlier phases while introducing runtime-specific protections that defend against attacks targeting deployed models.

A first group of mitigations restricts who can reach deployed models and what they can do once they have access, creating multiple layers of access control that constrain both external attackers and insider threats. Organizations can enforce strong authentication for production APIs and control model access [134] by restricting ML model and data access in production environments. One practical guidance set recommends strong access controls for LLM hosting platforms, code repositories, and training environments and stresses restricting LLM access to network resources, internal services, and APIs [84]. An Agentic AI Identity Management approach similarly emphasizes continuous verification for real time authentication, least privilege access, and micro segmentation to limit lateral movement [25]. Restricting model queries [159] protects against denial of ML service and excessive probing by limiting the number and rate of queries allowed for each model. At the network interface level, rate limiting and input validation and sanitization [85] on all proposed queries gate users from the actual model and reduce the chance of exploiting application logic or overwhelming the system with malicious requests. Network segmentation [188] further isolates production ML systems from other parts of the infrastructure, preventing attackers who compromise one system from easily pivoting to attack models. Filtering network traffic [196] blocks malicious communications and prevents data exfiltration by inspecting packets at network boundaries and enforcing policies about which services can communicate with deployed models. These access control and

network security measures work together to create a defense-in-depth posture where attackers must overcome multiple barriers to reach deployed models, and where each successful bypass still leaves them constrained by remaining controls.

Beyond access restrictions, monitoring and telemetry provide visibility into how deployed models are used and enable rapid detection of anomalous behavior that may indicate attacks or system failures. AI telemetry logging [170] and monitoring model queries in real time enables detection of suspicious patterns, potential exfiltration attempts, and misuse of privileged operations. An intelligent observability framework that combines AI telemetry and infrastructure metrics in a Kubernetes testbed reported a 35% improvement in anomaly detection accuracy and a reduction of mean time to resolution by more than 40% compared with baseline tools [213]. Traditional network security controls complement AI-specific monitoring. Network intrusion detection [199] systems, web application firewalls, and TLS-based secure communication for public-facing applications help prevent exploitation attempts. Additional practices such as input validation and sanitization, least privilege access to back end services, application isolation and sandboxing, exploit protection, network segmentation, and privileged account management provide multiple layers of control and visibility [30, 208]. Frequent vulnerability scans and timely software updates [189] further reduce exposure by identifying and patching weaknesses before they can be exploited. A case study on deep learning-based intrusion detection shows that deep neural networks can outperform other machine learning approaches for identifying network-based attacks [37]. These monitoring and traditional security controls work synergistically, with AI telemetry providing visibility into model-specific behaviors while network monitoring and intrusion detection catch broader infrastructure attacks, together creating comprehensive visibility across the entire production environment.

At the inference level, specialized defenses focus on the content of queries and responses to prevent adversarial manipulation of model behavior and protect against attacks that exploit model vulnerabilities through carefully crafted inputs. Adversarial input detection [130] evaluates incoming requests against known legitimate patterns and filters likely adversarial inputs before they reach the model, using statistical tests, learned classifiers, or consistency checks to identify suspicious queries. Model hardening techniques aim to make models less sensitive to small perturbations and more robust to adversarial attacks. In one benchmark, adversarial training for a deep neural network-based intrusion detection system improved robustness, achieving F1 scores of 93%, 99%, 85%, and 83% for benign, backdoor, ransomware, and cross-site scripting classes respectively [66]. Multi-modal sensors [162] can reduce risks in physical environments by combining diverse data sources to detect unauthorized access and tampering, particularly important for embodied AI systems such as autonomous vehicles or robotic systems that interact with the physical world. Pre-processing and input restoration techniques help counter physical perturbations such as adversarial stickers or projections that attempt to fool vision systems through modifications to the physical environment rather than digital inputs. These inference-level defenses create multiple checkpoints where adversarial inputs can be detected and rejected, hardened models that maintain correct behavior even when attacks succeed, and specialized protections for physical world scenarios where digital defenses alone are insufficient.

LLM deployments introduce unique security challenges that require additional specialized controls beyond traditional ML systems, particularly around prompt injection, jailbreaking, and the generation of harmful content. Generative AI guardrails [174] sit between user interactions and model outputs and can implement filters, rule-based logic, or classifier-based checks to enforce safety constraints. A recent control-theoretic guardrail approach goes beyond refusal and actively corrects risky outputs to safe ones, as demonstrated in simulated driving and e-commerce experiments [220]. Generative AI guidelines [175] define acceptable model behavior and are often embedded into system prompts, establishing boundaries around topics the model should not discuss or actions it should not take. LLM tagging [65] marks model outputs so that downstream components can identify their origin and distinguish between user-provided content and model-generated responses. Delimiting data and random sequence enclosure make it harder for prompts to be copied across interactions [65], while the

Table 2. Summary of MLOps mitigation strategies with empirical validation. Traditional IT column indicates defenses applicable to conventional systems (✓) or is unique to ML environments (×). The empirical validation column cites industry implementations and case studies demonstrating practical deployment and effectiveness of each technique.

Taxonomy	Mitigation Techniques	Empirical Validation	Traditional IT
Design and Setup	User Training [164]	Security Awareness Training (SAT) [60]	✓
	Strong Password Policies [192]	Global Info Study [215]	✓
	Multi-factor Authentication [191]	Azure AD Report [77]	✓
	Regular Audits [195]	Indonesia IT Audits [233]	✓
	Intrusion Prevention System [185]	Cisco case study [226]	✓
	Privilege Account Management [201]	Segura Privileged Access [234]	✓
	Secure Key Management	Google Cloud Key Management [46]	✓
	Encrypt Sensitive Data [187]	Thales Encryption [240]	✓
	Software Bill of Materials [34]	Forbes [20]	✓
	Code Signing [131]	AWS Lambda Signing [73]	×
	AI Bill of Materials [168]	Wiz AI-BOM [228]	×
	Cyber Persistence Detector [70]	CrowdStrike Detector [14]	✓
Model Development and Evaluations	Sanitize Training Data [160]	AWS Data Cleansing [8]	×
	Data and Model Lineage [178]	AWS Data and Model Lineage [24]	×
	Verify Artifacts [166]	Netflix [216]	×
	Encrypt Data and Model [35]	AWS Sagemaker [27]	✓
	Vulnerability Scanner [184]	ModelScan [222]	×
	Limit Release of Information [176]	×	×
	Regular Validations [258]	IBM Watson OpenScale [54]	×
	Adversarial Training [72]	Wiz AI-SPM [249]	×
	Use Ensemble Methods [161]	IBM [210]	×
	Model Hardening [152]	Avaly [12]	×
Operations	Adversarial Input Detection [130]	AWS [224]	×
	Input Validation and Sanitization [85]	Wiz [248]	×
	Restrict Model Queries [159]	Microsoft Azure ML [79]	×
	Control Model Access [134]	Microsoft Azure ML [79]	×
	Multi-Modal Sensors [162]	AWS [32]	×
	AI Telemetry Logging [170]	IBM Instana [230]	×
	LLM Tagging [65]	Appen [9]	×
	Generative AI Guidelines [175]	Google [44]	×
	Generative AI Guardrails [174]	AWS Bedrock Guardrails [6]	×
	Network Intrusion Detection [199]	Google Cloud IDS [45]	✓
	Network Segmentation [188]	Microsoft Azure ML [80]	✓
	Filter Network [196]	Microsoft [81]	✓
	Software Updates [189]	Apple Security Bounty [10]	✓

Sandwich Defense layers user instructions around previous responses, and Instruction Defense clarifies that models must not alter user inputs [65]. Marking prompts and outputs [65] further distinguishes user content from agent-generated content, helping to prevent prompt injection attacks where malicious instructions are hidden within seemingly benign inputs. Combined with guardrails and guidelines, these techniques reduce cross-model injection attempts and limit the spread of malicious prompts, although they also raise new security considerations if higher capability models that enforce the guardrails become compromised. Regular patching, developer and user education, thorough logging, and frequent audits close additional attack surfaces that attackers might otherwise exploit [18], while alignment techniques such as supervised fine-tuning and reinforcement learning from human feedback keep models aligned with organizational policy over time. These LLM-specific defenses address the unique attack surface created by natural language interfaces, where the boundary between data and instructions is fundamentally blurred and where models must distinguish between legitimate user requests and attempts to manipulate their behavior through carefully crafted prompts.

Across these three subsections we have outlined mitigation strategies that span the design and setup phase, the model development and evaluation phase, and the operations phase. Together, these defenses aim to protect MLOps and LLMOps deployments from user compromise and data gathering attacks through to model training, deployment, and runtime abuse. Table 2 summarizes the mitigation strategies discussed, their empirical validation through industry implementations, and their applicability beyond ML systems. However, the current mitigations remain fragmented and often reactive, and many combinations of tools and practices are not yet well understood in complex real world pipelines. In the next section we therefore turn to research recommendations, highlighting open challenges and directions for future work that can strengthen MLOps security across these interconnected domains and support more systematic lifecycle-aware defenses.

6 Research Challenges and Recommendation

Although the previous section outlined a range of mitigation strategies to defend the MLOps ecosystem, these measures often provide only partial protection against increasingly sophisticated, AI-driven adversaries. As attacks evolve rapidly, organizations must address not only technical vulnerabilities but also legal, ethical, and operational complexities. To address RQ4, this section identifies notable gaps that remain in current practices and proposes future research directions needed to strengthen security across the MLOps ecosystem. The following discussion highlights key concerns and proposes future research directions, with the goal of strengthening the security of the MLOps ecosystem while preserving essential principles of transparency, privacy, and compliance.

- (1) **The Rise of AI-Driven Social Engineering:** Attackers now exploit generative AI to craft highly convincing phishing emails, clone voices for vishing schemes, and produce deepfake videos that entice employees into revealing sensitive credentials [33]. Once inside an organization, these adversaries can move laterally and compromise vital MLOps components. Although zero trust models and ongoing user education are vital, they frequently fall behind AI's rapid advancement. Further research should refine deepfake detection techniques, enhance digital authentication protocols, and implement comprehensive incident monitoring to counter the increasingly realistic nature of AI-enabled social engineering.
- (2) **Malicious Repositories and LLM Hallucinations:** LLMs occasionally generate misleading "hallucinations", which can enable attackers to insert malicious packages into trusted online repositories. If organizations inadvertently integrate these compromised resources, they risk exposing intellectual property, disrupting workflows, or introducing harmful payloads. Threat actors may also upload cloned or tampered models to platforms such as Hugging Face, deceiving unsuspecting practitioners into deploying compromised versions [245]. Future research should explore automated systems for detecting suspicious content, robust versioning protocols to ensure code integrity, and community-driven validation frameworks supported by the broader MLOps ecosystem.

- (3) **Misuse of AI Tools by Advanced Persistent Threats (APTs):** Advanced Persistent Threats (APTs) groups sponsored by nation state actors increasingly rely on freely available powerful AI tools, such as Google's Gemini, to identify vulnerabilities, develop sophisticated malware, and evade standard security measures [48]. Google DeepMind's Gemma team has raised concerns about potential misuse of AI in upcoming releases [42]. Researchers should examine subtle ways AI may be weaponized, establish precise ethical and legal standards, and propose robust prevention strategies. Effectively defending MLOps ecosystem against increasingly sophisticated adversaries will require detecting and preventing abuse of emerging AI technologies.
- (4) **Benchmarking MLOps Defenses:** Many firms rely on laboratory-style tests that do not accurately reflect real-world attacks. As a result, MLOps ecosystem remain vulnerable when deployed in practical settings. Future research should focus on benchmarking defenses against adaptive and diverse attack patterns [243]. By simulating adversarial scenarios, such as targeted phishing attempts and sophisticated AI-powered attacks, MLOps teams can evaluate how well their security measures perform under pressure. Additionally, continuous research should establish standardized criteria for comparing defensive mechanisms. This approach will support ongoing improvement and ensure better alignment with real operational risks.
- (5) **Red-Teaming for Robust Security:** Red team exercises serve as an effective approach for uncovering hidden vulnerabilities, validating existing defenses, and strengthening overall system resilience. Drawing on initiatives such as MITRE's red team program, organizations can more accurately simulate real-world attack scenarios and identify exploit paths that may remain undetected during conventional testing. Future research should explore frameworks that support recurring, collaborative red-teaming efforts across sectors, ensuring that shared insights contribute to a more secure global MLOps ecosystem. By embracing these rigorous assessments, security teams can enhance threat detection, improve incident response, and cultivate a proactive security posture.
- (6) **Strengthening MLOps Robustness:** A robust MLOps strategy adopts security as a default posture and embeds high-assurance practices throughout the ML lifecycle. Regular testing using established adversarial simulation tools and tailored threat scenarios can help uncover and remediate vulnerabilities before they lead to critical failures. Further investigation is needed to identify optimal strategies for streamlining configuration management, continuous patching, and resilience assessments across the MLOps ecosystem. This proactive approach enables organizations to anticipate, disrupt, and adapt to evolving adversarial threats.
- (7) **Open Source vs. Closed Source:** An important question in securing MLOps is how to balance openness with security when comparing closed-source and open-source approaches. Publicly accessible code fosters collaborative innovation but may also provide adversaries with insights into system architecture. Conversely, limited disclosure can hinder attackers but may reduce transparency and erode user trust. Further research should explore how community-driven governance, selective disclosure, and cryptographic verification can preserve the benefits of open development while reducing reconnaissance risks. By striking this balance, organizations can maintain stakeholder trust and safeguard operational integrity.
- (8) **Cultivating Cyber-Hygiene Among MLOps Practitioners:** Developers, DevOps engineers, and data scientists often work in silos, unintentionally introducing security risks in MLOps environments. While password encryption and secure networking are crucial, practitioners also need training in secure coding [164], proactive vulnerability management, and regular data backups. The compelling and clear evidence for this is that Effective security awareness training, with regular simulated phishing exercises, educates employees and significantly reduces the human risk of cybersecurity threats [60]. Further research is needed to assess the effectiveness of training methods, incentive structures, and policies in fostering a security-oriented culture at both organizational and societal levels.

- (9) **Balancing Transparency with Trust:** Organizations often publish white papers and technical reports to showcase their commitment to ethical and responsible AI. However, excessive disclosure of MLOps infrastructure can provide adversaries with a blueprint for exploitation. Researchers must develop frameworks that promote transparency in fairness criteria and privacy protections without revealing sensitive operational details. Future research should focus on defining indicators of responsible AI that offer public and regulatory assurance without compromising system security. A well-balanced transparency model can help organizations build trust while safeguarding confidential information.
- (10) **The Challenge of Copyright, Privacy, and Compliance:** Generative AI models may unintentionally expose copyrighted or sensitive personal data, leading to significant legal consequences and reputational damage. When models continuously train on real or user-provided data, issuing disclaimers provides only a minimal level of protection [56]. Scholars should develop comprehensive solutions that include frequent model audits, automated data validation, and tightly controlled input and output streams. These measures can help AI advancements comply with legal requirements while safeguarding user privacy and intellectual property. Recent work on model watermarking suggests that such techniques can contribute to post hoc copyright enforcement by enabling the detection of stolen models, yet they remain a passive defense whose current designs are mostly evaluated on small scale image classification tasks, leaving open questions about scalability and applicability to other machine learning domains before they can be broadly adopted in practice [17]. Future work should strengthen the stability of watermarking schemes, integrate them with active security controls, extend them beyond small image classification settings to diverse data types and learning tasks, and align their use with real world legal and organizational processes for demonstrating and enforcing ownership claims [17].

Protecting MLOps ecosystem requires a holistic strategy that combines technological solutions with organizational best practices. By embracing rigorous benchmarking, conducting thorough red-team assessments, and establishing frameworks that prioritize security from the onset, organizations can better defend their MLOps ecosystems against emerging attacks.

7 Conclusion

In recent years, MLOps has fundamentally reshaped how machine learning models are developed, deployed, and maintained. It offers substantial advantages, including enhanced scalability, reproducibility, rapid deployment, and improved team collaboration. However, the acceleration of development to deployment cycles, combined with increased automation and highly interconnected workflows, significantly expands the attack surface. This heightens the risk of vulnerabilities and security breaches. As machine learning becomes increasingly embedded in critical systems, these trends underscore the urgent need for cybersecurity frameworks that can evolve alongside technological advancements.

This study highlights that while MLOps delivers critical operational benefits, it also introduces security challenges that demand focused attention. Leveraging the MITRE ATLAS framework for AI-specific threats, we present a structured taxonomy of vulnerabilities spanning the entire MLOps lifecycle, from data collection to deployment. Insights from red-team exercises and real-world incidents illustrate how adversaries exploit both user-level and system-level weaknesses. To address these threats, we propose targeted mitigation strategies informed by established frameworks and best practices. These recommendations offer guidance for improving MLOps security and identifying priorities for future research.

As MLOps adoption continues to grow, organizations must embrace a proactive security posture by embedding robust practices early in their machine learning workflows. By implementing the frameworks and recommendations outlined in this work, they can more effectively safeguard models, maintain stakeholder and customer confidence, and ensure that machine learning continues to support secure, ethical, and responsible innovation. In

doing so, they establish a resilient foundation for long-term security, sustained innovation, and adaptive growth in a rapidly evolving threat landscape.

Acknowledgment

This work was supported by the Predictive Analytics and Technology Integration (PATENT) Laboratory. The author gratefully acknowledges the valuable feedback, discussions, and support provided by the laboratory members, which contributed to improving the quality of this paper.

References

- [1] 4N7M4N. Careful Who You Colab With: Abusing Google Colaboratory. <https://antman1p-30185.medium.com/careful-who-you-colab-with-fa8001f933e7>, Jul 2022. Accessed: 2025-01-26.
- [2] AHAKONYE, L. A. C., AMAIZU, G. C., NWAKANMA, C. I., LEE, J. M., AND KIM, D. S. Enhanced vulnerability detection in scada systems using hyper-parameter-tuned ensemble learning. In *International Conference on ICT Convergence* (2021), vol. 2021-October, IEEE Computer Society, pp. 458–461.
- [3] AHMAD, T., ADNAN, M., RAFI, S., AKBAR, M. A., AND ANWAR, A. Mlops-enabled security strategies for next-generation operational technologies. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering* (New York, NY, USA, 2024), EASE '24, Association for Computing Machinery, p. 662–667.
- [4] ALEXEY ANTONOV AND ALEXEY KOGTENKOV. How to confuse antimalware neural networks. Adversarial attacks and protection. <https://securelist.com/how-to-confuse-antimalware-neural-networks-adversarial-attacks-and-protection/102949/>, Jun 2021. Accessed: 2025-01-26.
- [5] ALI, A., PINCIROLI, R., YAN, F., AND SMIRNI, E. BATCH: Machine Learning Inference Serving on Serverless Platforms with Adaptive Batching. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis* (2020), pp. 1–15.
- [6] AMAZON WEB SERVICES. Amazon bedrock guardrails. Accessed: 2025-11-19.
- [7] AMAZON WEB SERVICES. AWS Identity and Access Management. <https://aws.amazon.com/iam/>. Accessed: 2024-7-14.
- [8] AMAZON WEB SERVICES. What is Data Cleansing? <https://aws.amazon.com/what-is/data-cleansing/>. Accessed: 2025-11-09.
- [9] APPEN. Data annotation services for AI and ML models. Accessed: 2025-11-19.
- [10] APPLE INC. Apple Security Bounty. <https://security.apple.com/bounty/>, 2025. Accessed: 2025-11-11.
- [11] ATLISSIAN. Trello — Project Management Software. <https://trello.com/>, 2024. Accessed: 2024-12-16.
- [12] AVALY AI. Robust model hardening. Accessed: 2025-11-19.
- [13] AX SHARMA. PyTorch discloses malicious dependency chain compromise over holidays, Jan 2023. Accessed: 2025-01-26.
- [14] BAKER, K. ADVANCED PERSISTENT THREATS (APT) EXPLAINED. <https://www.crowdstrike.com/en-us/cybersecurity-101/threat-intelligence/advanced-persistent-threat-apt/>, Mar. 2024. Accessed: 2025-11-10.
- [15] BALANTRAPU, S. S. Adversarial Machine Learning: Security Threats and Mitigations. *International Journal of Sustainable Development in Computing Science* 1, 3 (2019), 1–18.
- [16] BISONG, EKABA. *Google Colaboratory*. Apress, Berkeley, CA, 2019.
- [17] BOENISCH, F. A Systematic Review on Model Watermarking for Neural Networks. *Frontiers in Big Data Volume 4 - 2021* (2021).
- [18] BRANESCU, I., GRIGORESCU, O., AND DASCALU, M. Automated Mapping of Common Vulnerabilities and Exposures to MITRE ATT&CK Tactics. *Information* 15, 4 (2024), 214.
- [19] BREWER, ERIC A. Kubernetes and the path to cloud native. In *Proceedings of the Sixth ACM Symposium on Cloud Computing* (New York, NY, USA, 2015), SoCC '15, Association for Computing Machinery, p. 167.
- [20] BRIDGWATER, A. Never drop the sbom, why a software bill of materials pays off. <https://www.forbes.com/sites/adrianbridgwater/2025/09/18/never-drop-the-sbom-why-a-software-bill-of-materials-pays-off/>, Sep 2025. Accessed: 2025-11-10.
- [21] CALEFATO, F., LANUBILE, F., AND QUARANTA, L. Security Risks and Best Practices of MLOps: A Multivocal Literature Review. In *The Italian Conference on CyberSecurity (ITASEC 2024)* (Apr. 2024), Zenodo.
- [22] CARLINI, N., JAGIELSKI, M., CHOQUETTE-CHOO, C. A., PALEKA, D., PEARCE, W., ANDERSON, H., TERZIS, A., THOMAS, K., AND TRAMÈR, F. Poisoning Web-Scale Training Datasets is Practical. In *2024 IEEE Symposium on Security and Privacy (SP)* (2024), pp. 407–425.
- [23] CAZORLA, L., ALCARAZ, C., AND LOPEZ, J. Cyber Stealth Attacks in Critical Information Infrastructures. *IEEE Systems Journal* (2018).
- [24] CHRISTENSEN, H., BENSON, D., THADAKA, K., TRIKANDE, S., AND KRISHNAN, V. Model and data lineage in machine learning experimentation. <https://aws.amazon.com/blogs/machine-learning/model-and-data-lineage-in-machine-learning-experimentation/>, Sept. 2021. Accessed: 2025-11-10.
- [25] CLOUD SECURITY ALLIANCE (CSA). Agentic AI Identity Management Approach. <https://cloudsecurityalliance.org/blog/2025/03/11/agentic-ai-identity-management-approach#>. Accessed: 2025-11-09.

- [26] COMPUTRONIX. AI Phishing Attacks: How Cybercriminals Use Automation to Target You. <https://computronixusa.com/ai-phishing-attacks-cybercriminals-leveraging-automation/>, 2025. Accessed: 2025-01-26.
- [27] D'ALIBERTI, L., KOVBA, J., GULATI, M., HODA, S., AND SNIVELY, B. Enable fully homomorphic encryption with Amazon SageMaker endpoints for secure, real-time inferencing. <https://aws.amazon.com/blogs/machine-learning/enable-fully-homomorphic-encryption-with-amazon-sagemaker-endpoints-for-secure-real-time-inferencing/>, March 2023. Accessed: 2025-11-09.
- [28] DANIEL HUYNH AND JADE HARDOUIN. PoisonGPT: How We Hid a Lobotomized LLM on Hugging Face to Spread Fake News. <https://blog.mithrilsecurity.io/poisongpt-how-we-hid-a-lobotomized-llm-on-hugging-face-to-spread-fake-news/>, Jul 2023. Accessed: 2025-02-05.
- [29] DEEPSEEK-AI TEAM. DeepSeek-V3 Technical Report. <https://arxiv.org/abs/2412.19437>, 2024.
- [30] DING, M. Analyzing the Research and Application of the RSA Cryptosystem in the Context of Digital Signatures. *Transactions on Computer Science and Intelligent Systems Research* 5 (Aug 2024), 26–34.
- [31] EKEN, B., PALLEWATTA, S., TRAN, N., TOSUN, A., AND BABAR, M. A. A Multivocal Review of MLOps Practices, Challenges and Open Issues. *ACM Comput. Surv.* 58, 2 (Sept. 2025).
- [32] ELAPROLU, S., SIMONEAU, A., AMADEO, P., AND LARSON, R. Transforming the physical world with AI: The next frontier in intelligent automation, October 2025. Accessed: 2025-11-19.
- [33] EMILY ASTRANOVA AND PASCAL ISSA. AI-Powered Voice Spoofing for Next-Gen Vishing Attacks. <https://cloud.google.com/blog/topics/threat-intelligence/ai-powered-voice-spoofing-vishing-attacks>, Jul 2024. Accessed: 2025-01-29.
- [34] ENDURING SECURITY FRAMEWORK. Securing the Software Supply Chain: Recommended Practices for Software Bill of Materials Consumption. <https://media.defense.gov/2023/Nov/09/2003338086/-1/-1/0/SECURING%20THE%20SOFTWARE%20SUPPLY%20CHAIN%20RECOMMENDED%20PRACTICES%20FOR%20SOFTWARE%20BILL%20OF%20MATERIALS%20CONSUMPTION.PDF>, nov 2023. Accessed: 2025-11-09.
- [35] ESMRADI, A., YIP, D. W., AND CHAN, C. F. A Comprehensive Survey of Attack Techniques, Implementation, and Mitigation Strategies in Large Language Models. In *International Conference on Ubiquitous Security* (2023), Springer, pp. 76–95.
- [36] ET AL., N. P. Technical Report on the CleverHans v2.1.0 Adversarial Examples Library. *arXiv preprint arXiv:1610.00768* (2018).
- [37] FERNÁNDEZ, G. C., AND XU, S. A case study on using deep learning for network intrusion detection. In *MILCOM 2019 - 2019 IEEE Military Communications Conference (MILCOM)* (2019), pp. 1–6.
- [38] FERRY, N., ROSSINI, A., CHAUVEL, F., MORIN, B., AND SOLBERG, A. Towards Model-Driven Provisioning, Deployment, Monitoring, and Adaptation of Multi-cloud Systems. In *2013 IEEE Sixth International Conference on Cloud Computing* (2013), pp. 887–894.
- [39] FRANK MCKENNA. The Many Jobs and Wigs of Eric Jaklitch's Fraud Scheme. <https://frankonfraud.com/the-many-jobs-and-wigs-of-eric-jaklitchs-fraud-scheme/>, Jan 2022. Accessed: 2025-02-02.
- [40] GAN, A., AND GHODSI, Z. Sentry: Authenticating machine learning artifacts on the fly. <http://arxiv.org/abs/2510.00554>, 10 2025.
- [41] GARG, S., PUNDIR, P., RATHEE, G., GUPTA, P., GARG, S., AND AHLAWAT, S. On Continuous Integration / Continuous Delivery for Automated Deployment of Machine Learning Models using MLOps. In *2021 IEEE Fourth International Conference on Artificial Intelligence and Knowledge Engineering (AIKE)* (2021), pp. 25–28.
- [42] GEMMA TEAM ET. AL. Gemma: Open Models Based on Gemini Research and Technology, 2024.
- [43] GOLDBLUM, M., TSIPRAS, D., XIE, C., CHEN, X., SCHWARZSCHILD, A., SONG, D., MADRY, A., LI, B., AND GOLDSTEIN, T. Dataset Security for Machine Learning: Data Poisoning, Backdoor Attacks, and Defenses. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2023).
- [44] GOOGLE. Safety guidance. Accessed: 2025-11-19.
- [45] GOOGLE CLOUD. Cloud intrusion detection system, 2025. Accessed: 2025-11-19.
- [46] GOOGLE CLOUD. Cloud Key Management Service encryption. <https://docs.cloud.google.com/docs/security/key-management-deep-dive>, Jan 2025. Accessed: 2025-11-09.
- [47] GOOGLE CLOUD. What is LLMops (large language model operations)? <https://cloud.google.com/discover/what-is-llmops>, 2025. Accessed: 2025-11-03.
- [48] GOOGLE THREAT INTELLIGENCE GROUP. Adversarial Misuse of Generative AI. <https://cloud.google.com/blog/topics/threat-intelligence/adversarial-misuse-generative-ai>, Jan 2025. Accessed: 2025-01-29.
- [49] GRAND VIEW RESEARCH. MLOps Market (2025 - 2030). <https://www.grandviewresearch.com/industry-analysis/mlops-market-report>, 2024.
- [50] GRANGER, B. E., AND PÉREZ, F. Jupyter: Thinking and Storytelling With Code and Data. *Computing in Science & Engineering* (2021), 7–14.
- [51] GUIDO VAN ROSSUM. Python Tutorial. Tech. Rep. CS-R9526, Centrum voor Wiskunde en Informatica (CWI), Amsterdam, May 1995.
- [52] HAWKINS, B., AND THOMPSON, C. Abusing mlops platforms to compromise ml models and enterprise data lakes. Tech. rep., IBM X-Force Red, Nov. 2024. Accessed: 2025-11-08.
- [53] HENDRYCKS, D., BURNS, C., KADAVATH, S., ARORA, A., BASART, S., TANG, E., SONG, D., AND STEINHARDT, J. Measuring Mathematical Problem Solving With the MATH Dataset, 2021.

- [54] IBM. Manage Model Risk. <https://dataplatfom.cloud.ibm.com/docs/content/wsj/model/cloud-risk-wos-only.html?context=cpdaas>, 2025. Accessed: 2025-11-19.
- [55] IBM RESEARCH. Adversarial Robustness Toolbox. <https://adversarial-robustness-toolbox.org/>, Jul 2020.
- [56] ILIA KOLOCHENKO AND GORDON PLATT. Data Security Professional Perspective: ChatGPT IP Cybersecurity. <https://www.bloomberglaw.com/external/document/X7K3GI38000000/data-security-professional-perspective-chatgpt-ip-cybersecurity->, Feb 2023. Accessed: 2025-01-29.
- [57] JARAMILLO, L. E. S. Malware detection and mitigation techniques: Lessons learned from Mirai DDOS attack. *Journal of Information Systems Engineering & Management* 3, 3 (2018), 19.
- [58] JAREK KAZMIERCZAK AND KHALID SALAMA AND VALENTIN HUERTA AND SUNIL KUMAR JANG BAHADUR. MLOps: Continuous delivery and automation pipelines in machine learning, Aug 2024.
- [59] JOHN, M. M., OLSSON, H. H., AND BOSCH, J. Towards MLOps: A Framework and Maturity Model. In *2021 47th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)* (2021), pp. 1–8.
- [60] KNOWBE4. Knowbe4 research confirms effective security awareness training significantly reduces data breaches. <https://www.knowbe4.com/press/knowbe4-research-confirms-effective-security-awareness-training-significantly-reduces-data-breaches>, 2025. Accessed: 2025-11-09.
- [61] KOIZUMI, R., AND SASAKI, R. Study on Countermeasures Using Mitigation Software against Vulnerability Attacks. In *2015 Fourth International Conference on Cyber Security, Cyber Warfare, and Digital Forensic (CyberSec)* (2015), IEEE, pp. 28–33.
- [62] KREUZBERGER, D., KÜHL, N., AND HIRSCHL, S. Machine learning operations (MLOps): Overview, Definition, and Architecture. *IEEE Access* 11 (2023), 31866–31879.
- [63] KUMARA, I., ARTS, R., NUCCI, D., HEUVEL, W., AND TAMBURRI, D. Requirements and Reference Architecture for MLOps: Insights from Industry.
- [64] KURT BAKER. Open Source Intelligence (OSINT). <https://www.crowdstrike.com/en-us/cybersecurity-101/threat-intelligence/open-source-intelligence-osint/>, Jan 2025. Accessed: 2025-02-01.
- [65] LEE, D., AND TIWARI, M. Prompt Infection: LLM-to-LLM Prompt Injection within Multi-Agent Systems, 2024.
- [66] LELLA, E., MACCHIARULO, N., PAZIENZA, A., LOFÜ, D., ABBATECOLA, A., AND NOVIELLO, P. Improving the robustness of dnns-based network intrusion detection systems through adversarial training. In *2023 8th International Conference on Smart and Sustainable Technologies (SpliTech)* (2023), pp. 1–6.
- [67] LI, Y., HUA, J., WANG, H., CHEN, C., AND LIU, Y. DeepPayload: Black-box Backdoor Attack on Deep Learning Models through Neural Payload Injection, 2021.
- [68] LI, Y., LYU, X., KOREN, N., LYU, L., LI, B., AND MA, X. Anti-backdoor learning: Training clean models on poisoned data. In *NeurIPS* (2021).
- [69] LIMA, A., MONTEIRO, L., AND FURTADO, A. P. MLOps: Practices, Maturity Models, Roles, Tools, and Challenges - A Systematic Literature Review. In *Proceedings of the 24th International Conference on Enterprise Information Systems - Volume 2: ICEIS*, (2022), vol. 1, Science and Technology Publications, Lda.
- [70] LIU, Q., SHOAIB, M., REHMAN, M. U., BAO, K., HAGENMEYER, V., AND HASSAN, W. U. Accurate and Scalable Detection and Investigation of Cyber Persistence Threats, 2024.
- [71] LUMELSKY, A., KAPLAN, G., AND ELBAZ, G. ShadowRay: First Known Attack Campaign Targeting AI Workloads Actively Exploited In The Wild. <https://www.oligo.security/blog/shadowray-attack-ai-workloads-actively-exploited-in-the-wild>, Mar 2024. Accessed: 2025-01-11.
- [72] MA, L., AND LIANG, L. Increasing-margin adversarial (ima) training to improve adversarial robustness of neural networks. *Computer Methods and Programs in Biomedicine* 240 (10 2023).
- [73] MARTIN, C. Best practices and advanced patterns for lambda code signing. <https://aws.amazon.com/blogs/security/best-practices-and-advanced-patterns-for-lambda-code-signing/>, Jan 2021. Accessed: 2025-11-09.
- [74] MATTHEW GAULT. OpenAI Claims DeepSeek Plagiarized Its Plagiarism Machine. <https://gizmodo.com/openai-claims-deepseek-plagiarized-its-plagiarism-machine-2000556339>, Jan 2025. Accessed: 2025-02-23.
- [75] MEHMOOD, M., AMIN, R., MUSLAM, M. M. A., XIE, J., AND ALDABBAS, H. Privilege Escalation Attack Detection and Mitigation in Cloud Using Machine Learning. *IEEE Access* 11 (2023), 46561–46576.
- [76] MERKEL, DIRK. Docker: lightweight Linux containers for consistent development and deployment. *Linux J.* 2014, 239 (3 2014).
- [77] MEYER, L. A., ROMERO, S., BERTOLI, G., BURT, T., WEINERT, A., AND FERRES, J. L. How effective is multifactor authentication at deterring cyberattacks?
- [78] MIAO, Y., CHEN, C., PAN, L., HAN, Q.-L., ZHANG, J., AND XIANG, Y. Machine Learning-based Cyber Attacks Targeting on Controlled Information: A Survey. *ACM Computing Surveys (CSUR)* 54 (2021), 1–36.
- [79] MICROSOFT. Enterprise security and governance for Azure Machine Learning, October. Accessed: 2025-11-19.
- [80] MICROSOFT. Network isolation configurations for Azure Machine Learning. Accessed: 2025-11-19.
- [81] MICROSOFT. Virtual network service endpoints, 2025. Accessed: 2025-11-19.

- [82] MICROSOFT CORPORATION. Microsoft Azure. <https://azure.microsoft.com>. Accessed: 2025-02-02.
- [83] MICROSOFT CORPORATION. PowerShell. <https://learn.microsoft.com/en-us/powershell/>. Accessed: 2025-02-05.
- [84] MICROSOFT LEARN. Security planning for LLM-based applications. <https://learn.microsoft.com/en-us/ai/playbook/technology-guidance/generative-ai/mlops-in-openai/security/security-plan-llm-application>. Accessed: 2025-11-09.
- [85] MICROSOFT LEARN. Threat modeling ai/ml systems and dependencies. <https://learn.microsoft.com/en-us/security/engineering/threat-modeling-aiml>. Accessed: 2025-11-09.
- [86] MILLER, FREDERIC P. AND VANDOME, AGNES F. AND MCBREWSTER, JOHN. *Amazon Web Services*. Alpha Press, 2010.
- [87] MISHRA, J., AND SAHAY, S. K. Modern Hardware Security: A Review of Attacks and Countermeasures, 2025.
- [88] MITRE ATLAS. Face identification system evasion via physical countermeasures. <https://atlas.mitre.org/studies/AML.CS0012>, Jan 2020. Accessed: 2025-02-01.
- [89] MITRE ATLAS. Microsoft azure service disruption. <https://atlas.mitre.org/studies/AML.CS0010>, 2020. Accessed: 2025-02-01.
- [90] MITRE ATLAS. Virustotal poisoning. <https://atlas.mitre.org/studies/AML.CS0002>, 2020. Accessed: 2025-02-01.
- [91] MITRE ATLAS. Acquire infrastructure. <https://atlas.mitre.org/techniques/AML.T0008>, May 2021. Accessed: 2025-02-01.
- [92] MITRE ATLAS. Acquire infrastructure: ML development workspaces. <https://atlas.mitre.org/techniques/AML.T0008.000>, May 2021. Accessed: 2025-02-01.
- [93] MITRE ATLAS. Acquire public ml artifacts. <https://atlas.mitre.org/techniques/AML.T0002>, May 2021. Accessed: 2025-02-01.
- [94] MITRE ATLAS. Acquire public ml artifacts: Datasets. <https://atlas.mitre.org/techniques/AML.T0002.000>, 2021. Accessed: 2025-02-01.
- [95] MITRE ATLAS. Acquire public ml artifacts: Models. <https://atlas.mitre.org/techniques/AML.T0002.001>, 2021. Accessed: 2025-02-01.
- [96] MITRE ATLAS. Backdoor ml model. <https://atlas.mitre.org/techniques/AML.T0018>, May 2021. Accessed: 2025-02-01.
- [97] MITRE ATLAS. Cost harvesting. <https://atlas.mitre.org/techniques/AML.T0034>, May 2021. Accessed: 2025-02-01.
- [98] MITRE ATLAS. Craft adversarial data. <https://atlas.mitre.org/techniques/AML.T0043>, May 2021. Accessed: 2025-02-01.
- [99] MITRE ATLAS. Craft adversarial data: Insert backdoor trigger. <https://atlas.mitre.org/techniques/AML.T0043.004>, May 2021. Accessed: 2025-02-01.
- [100] MITRE ATLAS. Denial of ml service. <https://atlas.mitre.org/techniques/AML.T0029>, May 2021. Accessed: 2025-02-01.
- [101] MITRE ATLAS. Discover ml artifacts. <https://atlas.mitre.org/techniques/AML.T0007>, May 2021. Accessed: 2025-02-01.
- [102] MITRE ATLAS. Discover ml model family. <https://atlas.mitre.org/techniques/AML.T0014>, May 2021. Accessed: 2025-02-01.
- [103] MITRE ATLAS. Discover ml model ontology. <https://atlas.mitre.org/techniques/AML.T0013>, May 2021. Accessed: 2025-02-01.
- [104] MITRE ATLAS. Erode ml model integrity. <https://atlas.mitre.org/techniques/AML.T0031>, May 2021. Accessed: 2025-02-01.
- [105] MITRE ATLAS. Exfiltration via ml inference api. <https://atlas.mitre.org/techniques/AML.T0024>, May 2021. Accessed: 2025-02-01.
- [106] MITRE ATLAS. Exfiltration via ml inference api: Infer training data membership. <https://atlas.mitre.org/techniques/AML.T0024.000>, May 2021. Accessed: 2025-02-01.
- [107] MITRE ATLAS. Exfiltration viacyber means. <https://atlas.mitre.org/techniques/AML.T0025>, May 2021. Accessed: 2025-02-01.
- [108] MITRE ATLAS. Full ml model access. <https://atlas.mitre.org/techniques/AML.T0044>, May 2021. Accessed: 2025-02-01.
- [109] MITRE ATLAS. ML artifact collection. <https://atlas.mitre.org/techniques/AML.T0035>, May 2021. Accessed: 2025-02-01.
- [110] MITRE ATLAS. ML-enabled product or service. <https://atlas.mitre.org/techniques/AML.T0047>, May 2021. Accessed: 2025-02-01.
- [111] MITRE ATLAS. ML supply chain compromise, Oct 2021. Accessed: 2025-02-01.
- [112] MITRE ATLAS. ML supply chain compromise: Data. <https://atlas.mitre.org/techniques/AML.T0010.002>, May 2021. Accessed: 2025-02-01.
- [113] MITRE ATLAS. ML supply chain compromise: Hardware. <https://atlas.mitre.org/techniques/AML.T0010.00>, 2021. Accessed: 2025-02-01.
- [114] MITRE ATLAS. ML supply chain compromise: ML software. <https://atlas.mitre.org/techniques/AML.T0010.001>, 2021. Accessed: 2025-02-01.
- [115] MITRE ATLAS. ML supply chain compromise: Model. <https://atlas.mitre.org/techniques/AML.T0010.003>, May 2021. Accessed: 2025-02-01.
- [116] MITRE ATLAS. Physical environment access. <https://atlas.mitre.org/techniques/AML.T0041>, May 2021. Accessed: 2025-02-01.
- [117] MITRE ATLAS. Poison training data. <https://atlas.mitre.org/techniques/AML.T0020>, May 2021. Accessed: 2025-02-01.
- [118] MITRE ATLAS. Publish poisoned datasets. <https://atlas.mitre.org/techniques/AML.T0019>, May 2021. Accessed: 2025-02-01.
- [119] MITRE ATLAS. Search application repositories. <https://atlas.mitre.org/techniques/AML.T0004>, May 2021. Accessed: 2025-02-01.
- [120] MITRE ATLAS. Search for victim's publicly available research materials. <https://atlas.mitre.org/techniques/AML.T0000>, May 2021. Accessed: 2025-02-01.
- [121] MITRE ATLAS. Search for victim's publicly available research materials: Journals and conference proceedings. <https://atlas.mitre.org/techniques/AML.T0000.000>, May 2021. Accessed: 2025-02-01.
- [122] MITRE ATLAS. Search for victim's publicly available research materials: Pre-print repositories. <https://atlas.mitre.org/techniques/AML.T0000.001>, May 2021. Accessed: 2025-02-01.
- [123] MITRE ATLAS. Search for victim's publicly available research materials: Technical blogs. <https://atlas.mitre.org/techniques/AML.T0000.002>, May 2021. Accessed: 2025-02-01.
- [124] MITRE ATLAS. Search victim-owned websites. <https://atlas.mitre.org/techniques/AML.T0003>, May 2021. Accessed: 2025-02-01.

- [125] MITRE ATLAS. Spamming ml system with chaff data. <https://atlas.mitre.org/techniques/AML.T0046>, 2021. Accessed: 2025-02-01.
- [126] MITRE ATLAS. User execution: Unsafe ml artifacts. <https://atlas.mitre.org/techniques/AML.T0011.000>, 2021. Accessed: 2025-02-01.
- [127] MITRE ATLAS. Verify attack. <https://atlas.mitre.org/techniques/AML.T0042>, May 2021. Accessed: 2025-02-01.
- [128] MITRE ATLAS. Evade ml model. <https://atlas.mitre.org/techniques/AML.T0015>, Oct 2022. Accessed: 2025-02-01.
- [129] MITRE ATLAS. Active scanning. <https://atlas.mitre.org/techniques/AML.T0006>, Jan 2023. Accessed: 2025-02-01.
- [130] MITRE ATLAS. Adversarial input detection. <https://atlas.mitre.org/mitigations/AML.M0015>, Oct 2023. Accessed: 2025-02-01.
- [131] MITRE ATLAS. Code signing. <https://atlas.mitre.org/mitigations/AML.M0013>, Oct 2023. Accessed: 2025-02-01.
- [132] MITRE ATLAS. Collection. <https://atlas.mitre.org/tactics/AML.TA0009>, Jan 2023. Accessed: 2025-02-01.
- [133] MITRE ATLAS. Command and scripting interpreter. <https://atlas.mitre.org/techniques/AML.T0050>, Oct 2023. Accessed: 2025-02-01.
- [134] MITRE ATLAS. Control access to ml models and data at rest. <https://atlas.mitre.org/mitigations/AML.M0005>, 2023. Accessed: 2025-02-01.
- [135] MITRE ATLAS. Data from information repositories. <https://atlas.mitre.org/techniques/AML.T0036>, Jan 2023. Accessed: 2025-02-01.
- [136] MITRE ATLAS. Data from local system. <https://atlas.mitre.org/techniques/AML.T0037>, Jan 2023. Accessed: 2025-02-01.
- [137] MITRE ATLAS. Develop capabilities: Adversarial ml attacks. <https://atlas.mitre.org/techniques/AML.T0017.000>, 2023. Accessed: 2025-02-01.
- [138] MITRE ATLAS. Discovery. <https://atlas.mitre.org/tactics/AML.TA0008>, Jan 2023. Accessed: 2025-02-01.
- [139] MITRE ATLAS. Establish accounts. <https://atlas.mitre.org/techniques/AML.T0021>, Jan 2023. Accessed: 2025-02-01.
- [140] MITRE ATLAS. Exploit public-facing application. <https://atlas.mitre.org/techniques/AML.T0049>, Feb 2023. Accessed: 2025-02-01.
- [141] MITRE ATLAS. External harms. <https://atlas.mitre.org/techniques/AML.T0048>, Oct 2023. Accessed: 2025-02-01.
- [142] MITRE ATLAS. External harms: Ml intellectual property theft. <https://atlas.mitre.org/techniques/AML.T0048.004>, Oct 2023. Accessed: 2025-02-01.
- [143] MITRE ATLAS. External harms: Reputational harm. <https://atlas.mitre.org/techniques/AML.T0048.001>, Oct 2023. Accessed: 2025-02-01.
- [144] MITRE ATLAS. Impact. <https://atlas.mitre.org/tactics/AML.TA0011>, Jan 2023. Accessed: 2025-02-01.
- [145] MITRE ATLAS. Llm data leakage. <https://atlas.mitre.org/techniques/AML.T0057>, Oct 2023. Accessed: 2025-02-01.
- [146] MITRE ATLAS. Llm jailbreak. <https://atlas.mitre.org/techniques/AML.T0054>, Oct 2023. Accessed: 2025-02-01.
- [147] MITRE ATLAS. Llm meta prompt extraction. <https://atlas.mitre.org/techniques/AML.T0056>, Oct 2023. Accessed: 2025-02-01.
- [148] MITRE ATLAS. Llm plugin compromise. <https://atlas.mitre.org/techniques/AML.T0053>, Oct 2023. Accessed: 2025-02-01.
- [149] MITRE ATLAS. Llm prompt injection. <https://atlas.mitre.org/techniques/AML.T0051>, Oct 2023. Accessed: 2025-02-01.
- [150] MITRE ATLAS. Llm prompt injection: Direct. <https://atlas.mitre.org/techniques/AML.T0051.000>, Oct 2023. Accessed: 2025-02-01.
- [151] MITRE ATLAS. Llm prompt injection: Indirect. <https://atlas.mitre.org/techniques/AML.T0051.001>, Oct 2023. Accessed: 2025-02-01.
- [152] MITRE ATLAS. Model hardening. <https://atlas.mitre.org/mitigations/AML.M0003>, Oct 2023. Accessed: 2025-02-01.
- [153] MITRE ATLAS. Obtain capabilities. <https://atlas.mitre.org/techniques/AML.T0016>, 2023. Accessed: 2025-02-01.
- [154] MITRE ATLAS. Obtain capabilities: Software tools. <https://atlas.mitre.org/techniques/AML.T0016.001>, 2023. Accessed: 2025-02-01.
- [155] MITRE ATLAS. Persistence. <https://atlas.mitre.org/tactics/AML.TA0006>, Jan 2023. Accessed: 2025-02-01.
- [156] MITRE ATLAS. Phishing. <https://atlas.mitre.org/techniques/AML.T0052>, Oct 2023. Accessed: 2025-02-01.
- [157] MITRE ATLAS. Phishing: Spearphishing via social engineering llm. <https://atlas.mitre.org/techniques/AML.T0052.000>, Oct 2023. Accessed: 2025-02-01.
- [158] MITRE ATLAS. Privilege escalation. <https://atlas.mitre.org/tactics/AML.TA0012>, Oct 2023. Accessed: 2025-02-01.
- [159] MITRE ATLAS. Restrict number of ml model queries. <https://atlas.mitre.org/mitigations/AML.M0004>, Oct 2023. Accessed: 2025-02-01.
- [160] MITRE ATLAS. Sanitize training data. <https://atlas.mitre.org/mitigations/AML.M0007>, Oct 2023. Accessed: 2025-02-01.
- [161] MITRE ATLAS. Use ensemble methods. <https://atlas.mitre.org/mitigations/AML.M0006>, Oct 2023. Accessed: 2025-02-01.
- [162] MITRE ATLAS. Use multi-modal sensors. <https://atlas.mitre.org/mitigations/AML.M0009>, Oct 2023. Accessed: 2025-02-01.
- [163] MITRE ATLAS. User execution. <https://atlas.mitre.org/techniques/AML.T0011>, Jan 2023. Accessed: 2025-02-01.
- [164] MITRE ATLAS. User training. <https://atlas.mitre.org/mitigations/AML.M0018>, Oct 2023. Accessed: 2025-02-01.
- [165] MITRE ATLAS. Valid accounts. <https://atlas.mitre.org/techniques/AML.T0012>, Jan 2023. Accessed: 2025-02-01.
- [166] MITRE ATLAS. Verify ml artifacts. <https://atlas.mitre.org/mitigations/AML.M0014>, Oct 2023. Accessed: 2025-02-01.
- [167] MITRE ATLAS. Acquire infrastructure: Domains. <https://atlas.mitre.org/techniques/AML.T0008.002>, Oct 2024. Accessed: 2025-02-01.
- [168] MITRE ATLAS. Ai bill of materials. <https://atlas.mitre.org/mitigations/AML.M0023>, Oct 2024. Accessed: 2025-02-01.
- [169] MITRE ATLAS. Ai model inference api access. <https://atlas.mitre.org/techniques/AML.T0040>, Oct 2024. Accessed: 2025-02-01.
- [170] MITRE ATLAS. Ai telemetry logging. <https://atlas.mitre.org/mitigations/AML.M0024>, Oct 2024. Accessed: 2025-02-01.
- [171] MITRE ATLAS. Discover ai model outputs. <https://atlas.mitre.org/techniques/AML.T0063>, Oct 2024. Accessed: 2025-02-01.
- [172] MITRE ATLAS. Discover llm hallucinations. <https://atlas.mitre.org/techniques/AML.T0062>, Oct 2024. Accessed: 2025-02-01.
- [173] MITRE ATLAS. Erode dataset integrity. <https://atlas.mitre.org/techniques/AML.T0059>, Oct 2024. Accessed: 2025-02-01.
- [174] MITRE ATLAS. Generative ai guardrails. <https://atlas.mitre.org/mitigations/AML.M0020>, Oct 2024. Accessed: 2025-02-01.
- [175] MITRE ATLAS. Generative ai guidelines. <https://atlas.mitre.org/mitigations/AML.M0021>, Oct 2024. Accessed: 2025-02-01.

- [176] MITRE ATLAS. Limit public release of information. <https://atlas.mitre.org/mitigations/AML.M0000>, Oct 2024. Accessed: 2025-02-01.
- [177] MITRE ATLAS. Llm prompt self-replication. <https://atlas.mitre.org/techniques/AML.T0061>, Oct 2024. Accessed: 2025-02-01.
- [178] MITRE ATLAS. Model distribution methods. <https://atlas.mitre.org/mitigations/AML.M0017>, Jan 2024. Accessed: 2025-02-01.
- [179] MITRE ATLAS. Publish poisoned models. <https://atlas.mitre.org/techniques/AML.T0058>, Oct 2024. Accessed: 2025-02-01.
- [180] MITRE ATLAS. Reconnaissance. <https://atlas.mitre.org/tactics/AML.TA0002>, 2024. Accessed: 2025-02-01.
- [181] MITRE ATLAS. Search for publicly available adversarial vulnerability analysis. <https://atlas.mitre.org/techniques/AML.T0001>, Mar 2024. Accessed: 2025-02-01.
- [182] MITRE ATLAS. Unsecured credentials. <https://atlas.mitre.org/techniques/AML.T0055>, Apr 2024. Accessed: 2025-02-01.
- [183] MITRE ATLAS. User execution: Malicious package. <https://atlas.mitre.org/techniques/AML.T0011.001>, 2024. Accessed: 2025-02-01.
- [184] MITRE ATLAS. Vulnerability scanning. <https://atlas.mitre.org/mitigations/AML.M0016>, Jan 2024. Accessed: 2025-02-01.
- [185] MITRE ATT&CK. Behavior prevention on endpoint. <https://attack.mitre.org/mitigations/M1040/>, Jun 2019. Accessed: 2025-02-02.
- [186] MITRE ATT&CK. Credential access. <https://attack.mitre.org/tactics/TA0006/>, Jul 2019. Accessed: 2025-03-02.
- [187] MITRE ATT&CK. Encrypt sensitive information. <https://attack.mitre.org/mitigations/M1041/>, Jun 2019. Accessed: 2025-02-02.
- [188] MITRE ATT&CK. Network segmentation. <https://attack.mitre.org/mitigations/M1030/>, May 2020. Accessed: 2025-02-02.
- [189] MITRE ATT&CK. Update software. <https://attack.mitre.org/mitigations/M1051/>, Jul 2020. Accessed: 2025-02-02.
- [190] MITRE ATT&CK. Browser session hijacking. <https://attack.mitre.org/techniques/T1185/>, Feb 2022. Accessed: 2025-02-02.
- [191] MITRE ATT&CK. Multi-factor authentication. <https://attack.mitre.org/mitigations/M1032/>, Oct 2022. Accessed: 2025-02-02.
- [192] MITRE ATT&CK. Password policies. <https://attack.mitre.org/mitigations/M1027/>, Oct 2022. Accessed: 2025-02-02.
- [193] MITRE ATT&CK. Adversary-in-the-middle: Dhcp spoofing. <https://attack.mitre.org/techniques/T1557/003/>, 2024. Accessed: 2025-02-02.
- [194] MITRE ATT&CK. Application layer protocol: Web protocols. <https://attack.mitre.org/techniques/T1071/001/>, 2024. Accessed: 2025-02-02.
- [195] MITRE ATT&CK. Audit. <https://attack.mitre.org/mitigations/M1047/>, Oct 2024. Accessed: 2025-02-02.
- [196] MITRE ATT&CK. Filter network traffic. <https://attack.mitre.org/mitigations/M1037/>, Oct 2024.
- [197] MITRE ATT&CK. Forge web credentials. <https://attack.mitre.org/techniques/T1606/>, Oct 2024. Accessed: 2025-02-02.
- [198] MITRE ATT&CK. Network denial of service: Reflection amplification. <https://attack.mitre.org/techniques/T1498/002/>, Oct 2024. Accessed: 2025-02-02.
- [199] MITRE ATT&CK. Network intrusion prevention. <https://attack.mitre.org/mitigations/M1031/>, Oct 2024. Accessed: 2025-02-02.
- [200] MITRE ATT&CK. Phishing. <https://attack.mitre.org/techniques/T1566/>, Oct 2024. Accessed: 2025-02-02.
- [201] MITRE ATT&CK. Privileged account management. <https://attack.mitre.org/mitigations/M1026/>, Oct 2024. Accessed: 2025-02-02.
- [202] MITRE ATT&CK. Unsecured credentials. <https://attack.mitre.org/techniques/T1552/>, Oct 2024. Accessed: 2025-02-02.
- [203] MITRE ATT&CK. Unsecured credentials: Cloud instance metadata api. <https://attack.mitre.org/techniques/T1552/005/>, Oct 2024. Accessed: 2025-02-02.
- [204] MITRE ATT&CK. Valid accounts. <https://attack.mitre.org/techniques/T1078/>, Oct 2024. Accessed: 2025-02-02.
- [205] MITRE ATT&CK. Valid accounts: Cloud accounts. <https://attack.mitre.org/techniques/T1078/004/>, Oct 2024. Accessed: 2025-02-02.
- [206] MITRE ATT&CK. Valid Accounts: Default Accounts. <https://attack.mitre.org/techniques/T1078/001/>, Oct 2024. Accessed: 2025-02-02.
- [207] MITRE ATT&CK. Valid accounts: Local accounts. <https://attack.mitre.org/techniques/T1078/003/>, Oct 2024. Accessed: 2025-02-02.
- [208] MITRE CORPORATION. MITRE ATLAS. <https://atlas.mitre.org>. Accessed: 2024-07-27.
- [209] MITRE CORPORATION. MITRE ATT&CK Framework. <https://attack.mitre.org/>, 2024. Accessed: 2024-12-08.
- [210] MUREL, J., AND KAVLAKOGLU, E. What is ensemble learning?, 2023. Accessed: 2025-11-19.
- [211] MÄKINEN, S., SKOGSTRÖM, H., LAAKSONEN, E., AND MIKKONEN, T. Who Needs MLOps: What Data Scientists Seek to Accomplish and How Can MLOps Help? In *2021 IEEE/ACM 1st Workshop on AI Engineering - Software Engineering for AI (WAIN)* (2021), pp. 109–112.
- [212] NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. Development Operations (DevOps) - Glossary. https://csrc.nist.gov/glossary/term/development_operations. Accessed: 2025-04-10.
- [213] NIMMAGADDA, S. Applying ai/ml to kubernetes logging and monitoring in enhancing observability through intelligent systems. *European Journal of Computer Science and Information Technology* 13 (6 2025), 141–152.
- [214] NVIDIA. NVIDIA: MLPerf AI Benchmarks. <https://www.nvidia.com/en-us/data-center/resources/mlperf-benchmarks/>. Accessed: 2024-07-27.
- [215] O'HARA, P. Global information assurance certification paper implementing a stronger password policy. Tech. rep., SANS Institute, Sep 2002. Accessed: 2025-11-09.
- [216] OLADELE, S. How These 8 Companies Implement MLOps: In-Depth Guide. <https://neptune.ai/blog/how-these-8-companies-implement-mlops>, Sept. 2024. Accessed: 2025-11-10.
- [217] OSCAR SCHWARTZ. In 2016, Microsoft's Racist Chatbot Revealed the Dangers of Online Conversation, Nov 2019. Accessed: 2025-01-26.
- [218] OWASP. OWASP Top 10 for LLM Applications 2025. <https://owasp.org/www-project-top-10-for-large-language-model-applications/assets/PDF/OWASP-Top-10-for-LLMs-v2025.pdf>, Nov 2024. Accessed: 2025-01-11.
- [219] PANCHUMARTHI, S. Devsecmlps: A security framework for machine learning pipelines. *Authorea Preprints* (2025).

- [220] PANDYA, R., BLAND, M., NGUYEN, D. P., LIU, C., FISAC, J. F., AND BAJCSY, A. From refusal to recovery: A control-theoretic approach to generative ai guardrails.
- [221] PARMY OLSON. Faces Are the Next Target for Fraudsters. <https://www.wsj.com/articles/faces-are-the-next-target-for-fraudsters-11625662828>, Jul 2021. Accessed: 2025-02-02.
- [222] PROTECT AI. Modelscan. <https://protectai.com/modelscan>. Accessed: 2025-11-19.
- [223] RAUBER, J., ZIMMERMANN, R., BETHGE, M., AND BRENDL, W. Foolbox Native: Fast adversarial attacks to benchmark the robustness of machine learning models in PyTorch, TensorFlow, and JAX. *Journal of Open Source Software* 5, 53 (2020), 2607.
- [224] RAUSCHMAYR, N., AYDORE, S., KAYA, Y., AND ZAFAR, B. Detect adversarial inputs using Amazon SageMaker model monitor and Amazon SageMaker debugger, April 2022. Accessed: 2025-11-19.
- [225] REDDI, V. J. E. A. MLPerf Inference Benchmark. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)* (2020), pp. 446–459.
- [226] REID, G. Cisco protects data center assets with network-based intrusion prevention system. http://www.cisco.com/application/pdf/en/us/guest/products/ps6642/c1244/cdcont_0900aecd80313fac.pdf. Accessed: 2025-11-09.
- [227] REN, X., YE, Y., WU, X., WU, Y., AND XUE, Y. Demystifying the evolution of neural networks with bom analysis: Insights from a large-scale study of 55,997 github repositories, 9 2025.
- [228] ROTLEVI, S. AI-BOM: Building an AI Bill of Materials. <https://www.wiz.io/academy/ai-bom-ai-bill-of-materials>, Nov. 2025. Accessed: 2025-11-17.
- [229] RUBINSTEIN, B. I., NELSON, B., HUANG, L., JOSEPH, A. D., LAU, S.-H., RAO, S., TAFT, N., AND TYGAR, J. D. ANTIDOTE: understanding and defending against poisoning of anomaly detectors. In *Proceedings of the 9th ACM SIGCOMM Conference on Internet Measurement* (New York, NY, USA, 2009), IMC '09, Association for Computing Machinery, p. 1–14.
- [230] SALIFU, A. B., SOW, D., LIU, G., LEE, J., JHA, S., AND NARAYANASWAMI, C. Observe genai with ibm instana observability. Accessed: 2025-11-19.
- [231] SCULLEY, D., HOLT, G., GOLOVIN, D., DAVYDOV, E., PHILLIPS, T., EBNER, D., CHAUDHARY, V., YOUNG, M., CRESPO, J.-F., AND DENNISON, D. Hidden technical debt in Machine learning systems. In *Proceedings of the 29th International Conference on Neural Information Processing Systems* (Cambridge, MA, USA, 2015), vol. 2 of *NIPS'15*, MIT Press.
- [232] SEBASTIAN, G., AND KOLLURU, P. Rethinking the weakest link in the cybersecurity chain. *ISACA Journal* (2021). Accessed: 2025-02-02.
- [233] SEBAYANG, B. C., WILLIAM, C., MUDA, I., AND SEBAYANG, C. Implementation of information technology audit in improving company operational security and efficiency. Tech. rep.
- [234] SEGURA. Complete guide to privileged access management (pam). <https://segura.security/post/privileged-access-management-pam-a-complete-guide>. Accessed: 2025-11-09.
- [235] SHARBAT, T. Agentflayer: Discovery phase of AI agents in copilot studio. <https://labs.zenity.io/p/a-copilot-studio-story-discovery-phase-in-ai-agents-f917>, June 2025. Accessed: 2025-11-08.
- [236] SKYLIGHT CYBER. Cylance, I Kill You! <https://skylightcyber.com/2019/07/18/cylance-i-kill-you/>, Jul 2019. Accessed: 2025-02-01.
- [237] SLACK. Slack. <https://slack.com/>. Accessed: 2025-02-02.
- [238] STONE, J., PATEL, R., GHIASI, F., MITTAL, S., AND RAHIMI, S. Navigating MLOps: Insights into Maturity, Lifecycle, Tools, and Careers. In *2025 IEEE Conference on Artificial Intelligence (CAI)* (2025), pp. 643–650.
- [239] TESTI, M., BALLABIO, M., FRONTONI, E., IANNELLO, G., MOCCIA, S., SODA, P., AND VESSIO, G. MLOps: a taxonomy and a methodology. *IEEE Access* 10 (2022), 63606–63618.
- [240] THALES. The enterprise encryption blueprintdiscover, define and deploy your enterprise data encryption strategy. <https://dymarjaya.co.id/wp-content/uploads/2022/03/enterprise-encryption-blueprint-wp-v3.pdf>. Accessed: 2025-11-09.
- [241] TIM MUCCI AND COLE STRYKER. What is MLOps? | IBM. <https://www.ibm.com/think/topics/mlops>, Apr 2024. Accessed: 2025-04-10.
- [242] TOLPEGIN, V., TRUEX, S., GURSOY, M. E., AND LIU, L. Data poisoning attacks against federated learning systems. In *Computer security—ESORICS 2020: 25th European symposium on research in computer security, ESORICS 2020, guildford, UK, Sep 14–18, 2020, proceedings, part i 25* (2020), Springer, pp. 480–501.
- [243] VASSILEV, A., OPREA, A., FORDYCE, A., AND ANDERSON, H. Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations. Tech. Rep. NIST AI 100-2e2023, National Institute of Standards and Technology, Gaithersburg, MD, 2024.
- [244] VISENGERIEVA, L., KAMMER, A., BÄR, I., KNIESZ, A., AND PLÖD, M. Mlops principles. <https://ml-ops.org/content/mlops-principles>. Accessed: 2025-11-09.
- [245] VULCAN CYBER. Can you trust ChatGPT's package recommendations? <https://vulcan.io/blog/ai-hallucinations-package-risk>, Aug 2024. Accessed: 2025-01-26.
- [246] WALLACE, E., STERN, M., AND SONG, D. Imitation Attacks and Defenses for Black-box Machine Translation Systems, 2021.
- [247] WANG, L., LI, X., WANG, R., XIN, Y., GAO, M., AND CHEN, Y. Prenssem: A heterogeneous ensemble learning framework for vulnerability detection in software. *Applied Sciences (Switzerland)* 10 (11 2020), 1–17.
- [248] WIZ EXPERTS TEAM. Llm security for enterprises: Risks and best practices, October. Accessed: 2025-11-19.
- [249] WIZ EXPERTS TEAM. The threat of adversarial AI. <https://www.wiz.io/academy/adversarial-ai-machine-learning>, 2024. Accessed:

- 2025-11-19.
- [250] WUNDERWUZZI. ChatGPT Plugins: Data Exfiltration via Images & Cross Plugin Request Forgery. <https://embracethered.com/blog/posts/2023/chatgpt-webpilot-data-exfil-via-markdown-injection/>, May 2023. Accessed: 2025-01-26.
 - [251] YERLIKAYA, F. A., AND ŞERİF BAHTİYAR. Data poisoning attacks against machine learning algorithms. *Expert Systems with Applications* 208 (2022), 118101.
 - [252] YU, B., PAN, J., HU, J., NASCIMENTO, A., AND DE COCK, M. Character Level based Detection of DGA Domain Names. In *2018 International Joint Conference on Neural Networks (IJCNN)* (2018), pp. 1–8.
 - [253] ZACK WHITTAKER. Security lapse exposed Clearview AI source code. <https://techcrunch.com/2020/04/16/clearview-source-code-lapse/>, Apr 2020. Accessed: 2025-02-02.
 - [254] ZAKEER, S. Self-replicating Morris II worm targets ai email assistants. <https://securityintelligence.com/posts/morris-ii-self-replicating-malware-genai-email-assistants/>, Jun 2024.
 - [255] ZANKI, K. Malicious ML models discovered on Hugging Face platform. https://www.reversinglabs.com/blog/rl-identifies-malware-ml-model-hosted-on-hugging-face?web_view=true, Feb. 2025. Accessed: 2025-11-08.
 - [256] ZHANG, B. H., LEMOINE, B., AND MITCHELL, M. Mitigating Unwanted Biases with Adversarial Learning. In *Proceedings of the 2018 AAAI/ACM Conference on AI, Ethics, and Society* (2018), pp. 335–340.
 - [257] ZHANG, X., AND JASKOLKA, J. Conceptualizing the Secure Machine Learning Operations (SecMLOps) Paradigm. In *2022 IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS)* (2022), IEEE, pp. 127–138.
 - [258] ZHAO, J., AND LI, Q. Mitigating distribution shift in machine learning-augmented hybrid simulation, 6 2025.